
Part IV

Appendices

Astrophysical Recipes (Second Edition)

The art of AMUSE

Simon Portegies Zwart, Steve McMillan and Steven Rieder

Appendix A

AMUSE Fundamentals

You take the blue pill — the story ends, you wake up in your bed and believe whatever you want to believe. You take the red pill — you stay in Wonderland, and I show you how deep the rabbit hole goes.

—Morpheus

We present implementation details of the AMUSE framework. AMUSE contains more than 70 highly optimized codes and more than 5000 handy routines to manipulate data. We would love to discuss them all, but this would result in a horribly boring text to write, and read. We limit ourselves to describing the most important and most used methods, such as file I/O, data manipulation, how to tune the handling of units for diagnostics, and we list the most important parameters used in several of the domain-specific modules. We further list the most important packages currently interfaced with AMUSE.

A.1 Internal Parameters, Units, and Types

Most of the community modules in AMUSE have internal parameters that control the performance of the code. Some tune the accuracy or the details of the physics, while others inform the user of the state of the running module. Most are preset to reasonable values, often the values initially set by the original authors of the package, and for which the module has been validated. Under certain circumstances, you may want to change these parameters.

As discussed in Section 2.3.3, community codes can be queried for information on parameter settings, but even then some parameters stay hidden deep within the source code. Here, we present a brief overview of some of the parameters that appear in this book, and for which a reference may be needed.

Table A.1. Stellar Classifications in AMUSE

stellar_type	Physical Description
0	Fully convective low mass main-sequence star
1	Main-sequence star
2	Hertzsprung gap
3	First giant branch
4	Core-helium burning
5	First asymptotic giant branch
6	Second asymptotic giant branch
7	Main-sequence naked helium star
8	Hertzsprung gap naked helium star
9	Giant branch naked helium star
10	Helium white dwarf
11	Carbon/oxygen white dwarf
12	Oxygen/neon white dwarf
13	Neutron star
14	Black hole
15	Massless post-supernova object

A.1.1 Nontunable Parameters

There are a number of nontunable parameters in the AMUSE community codes. These often inform the user of the state of the underlying code. An example is the `stellar_type` parameter in the stellar evolution codes, as discussed in Section 5.3.12. The standard AMUSE stellar types are listed in Table A.1. In Section 5.2.2 we demonstrated how to query the stellar evolution type from a running stellar evolution code.

We have adopted the designation of stellar types used in SSE (Single-Star Evolution; Hurley et al. 2000). These designations are used to indicate the astrophysical state of a star, even though they are not 100% accurate—these types should be homogeneous across codes, but it is hard to guarantee consistency. While implementing the other stellar evolution codes, we attempted to port the SSE designations, but this did not always lead to a perfect classification. One example can be seen in Figure 5.3, where an asymptotic giant branch star is misidentified as a stellar remnant. Another is in MESA (Paxton et al. 2010, 2011, 2013, 2015, 2018; Gaburov et al. 2018), where the interface fails to report the core-helium burning stage even though this phase is actually properly modeled in the code.

A.1.2 Tunable Parameters

Many of the code-specific parameters can be tuned. Examples include the integration-controlling time-step parameter in gravity (see Section 4.3.6), metallicity of the star in stellar evolution (see Section 5.2.3), the adiabatic index (γ) in

hydrodynamics (see Section 6.2.3), or the number of photons in radiative transfer (see Chapter 7).

The following tables, Tables A.2 to A.6, present the tunable and nontunable parameters for the core models in gravity, stellar evolution, hydrodynamics, and radiative transfer.

In Table A.4, for example, we present an overview of the tunable parameters in the SPH code `Fi`. Some of them are quite similar to those used in other codes, such as `time_step`, `verbosity`, and `epsilon_squared`, which have similar meaning across disciplines. Others may sound somewhat obscure, but if you are an SPH expert, they actually make a lot of sense. Now, it is not our intention to turn the reader into an SPH expert, and we will explain a few of the more obscure names. We refer to Springel (2010b) for an excellent overview of the free parameters in SPH. Some physical parameters tune the behavior of the gas, rather than the numerics. Setting the viscosity of a gas (`alpha`) allows one to model various density domains, or gas compositions. Although it is a numerical parameter, its interpretation 234567890 is physical, but regrettably not calculable from first principles.

Let us just provide an example of the sort of discussion one of these parameters leads to. For example, the parameter `alpha` (a common name for a parameter in astrophysics) is an artificial viscosity parameter, with a typical value of 0.5. The parameter `beta`, between 0 and 1, refers to another artificial viscosity, often simply called the “beta parameter”, the value of which is $2 \times \alpha$, with a typical value of 1. Similarly, one might wonder why two artificial viscosity parameters, (`alpha` and `beta`) are needed, but numerically this seems to make sense.

Table A.2. Overview of the Most Common Tunable and Nontunable Gravity-Specific Parameters

Tunable Parameters			Gravitational Dynamics Explanation
Parameter Name	Value	Unit	
<code>begin_time</code>	0	Time	Star time for integrator
<code>epsilon_squared</code>	0	Length ²	Gravitational softening
<code>force_sync</code>	0	None	Force synchronization
<code>gpu_id</code>	−1	None	Identity of the GPU
<code>sync_time</code>	0	Time	...
<code>timestep_parameter</code>	0.14	None	Time-step parameter
<code>use_gpu</code>	0	Bool	Use the GPU
<code>zero_step_mode</code>	0	None	Do a zero time step
Nontunable Parameters			
Parameter Name	Value	Unit	Explanation
<code>block_steps</code>	0	None	Number of block-time steps
<code>gpu_id</code>	0	None	Identity of the GPU
<code>total_steps</code>	0	None	Number of integration steps

Table A.3. Overview of the Most Common Tunable and Nontunable Stellar Evolution Codes

Tunable Parameters			Stellar Evolution Explanation
Parameter Name	Value	Unit	
supernova_kick_velocity	600	km s ⁻¹	Supernova kick velocity
is_logging_of_evolve_enabled	False	Bool	Log stellar evolution
OB_wind_setting	1	Bool	O-star wind parameter
RGB_wind_setting	1	Bool	RGB wind parameter
AGB_wind_setting	1	Bool	AGB wind parameter
convective_overshoot_parameter	0.12	None	Convective overshoot length
import_model_entropy_accuracy	0.0001	None	Entropy solver accuracy
import_model_entropy_force	20	None	Entropy forcing efficiency
max_age_stop_condition	$2 \cdot 10^{12}$	yr	Stopping after a time
maximum_number_of_stars	10	None	Number of stars
metallicity	0.02	None	Stellar metalicity
min_timestep_stop_condition	10^6	s	Minimal time step
mixing_length_ratio	2	None	Mixing length parameter
number_of_ionization_elements	5	None	Number of ionizing elements
path_to_data	/data/input		Output directory path
semi_convection_efficiency	0.04	None	Convection efficiency
thermohaline_efficiency	1	None	Efficiency of the thermohalinety
verbosity	True	Bool	Maximal output

Table A.4. Overview of the Most Common Tunable and Nontunable Hydrodynamics Codes

Tunable Parameters Parameter Name	Hydrodynamics	
	Value	Unit
acc_timestep_crit_constant	0.25	None
artificial_viscosity_alpha	0.5	None
beta	1	None
code_length_unit	1	au
code_mass_unit	1	M _⊙
cool_par	1	None
courant	0.3	None
epsilon_squared	0	m ²
feedback	fuv	None
fi_data_directory	data/	input/
<code>_cell_opening_constant	0.01	None

gamma	5/3	None
gas_epsilon	0.005	au
max_density	100	$M_{\odot} \text{ au}^{-3}$
n_smooth	64	None
n_smooth_tol	0.1	None
nn_tol	0.1	None
opening_angle	0.5	None
optical_depth	0	None
periodic_box_size	1	au
sph_artificial_viscosity_eps	0.01	None
sph_h_const	0.2	au
sph_viscosity	sph	None
time step	58	Day
verbosity	0	None

We omit explaining the details of the parameter because most of them are rather detailed, require considerable explanation, and are probably hardly ever changed. The best way to learn what these parameters do is to look in the source code, and try them out. Just run a simple example script, such as `amuse.examples.stellar_minimal`, change one of the parameters and see what it does in the output. Some of the more obscure names include `beta`, `gamma`, `courant`, `feedback`, `heat_par1`, `heat_par2`.

Table A.5. Overview of the Boolean Parameters in SPH Codes

Boolean Value	Boolean Parameter Name
True	<code>acc_timestep_flag</code> , <code>conservative_sph_flag</code> , <code>eps_is_h_flag</code> , <code>gadget_cell_opening_flag</code> , <code>integrate_entropy_flag</code> , <code>sph_dens_init_flag</code> , <code>use_hydro_flag</code> , <code>self_gravity_flag</code> .
False	<code>adaptive_smoothing_flag</code> , <code>balsara_flag</code> , <code>direct_sum_flag</code> , <code>enforce_min_sph_grav_softening_flag</code> , <code>fixed_halo_flag</code> , <code>freeform_timestep_flag</code> , <code>freeform_timestep_flag</code> , <code>isothermal_flag</code> , <code>periodic_boundaries_flag</code> , <code>quadrupole_moments_flag</code> , <code>radiation_flag</code> , <code>smooth_input_flag</code> , <code>square_root_timestep_flag</code> , <code>star_formation_flag</code> .

Table A.6. Overview of the Most Common Tunable and Nontunable Radiative Transfer Specific Parameters

Tunable Parameters			Radiative Transfer Explanation
Parameter Name	Value	Unit	
boundary_condition	0	None	See below
box_size	13	kpc	Box size, if bound
default_spectral_type	−1	None	Radiation spectrum
helium_case_A_flag	True	Bool	Helium recombination
hydrogen_case_A_flag	True	Bool	Hydrogen recombination
isothermal_flag	False	bool	Evolve temperature or not
momentum_kicks_flag	False	Bool	Use the momentum kick
number_of_rays	1000	yr ^{−1}	Number of rays
spectral_file	./spectra/thermal30k.cdf		

Another example is (as you might guess) gamma. This parameter has a completely different meaning from alpha and beta, and its effect on the computational results is profound. The parameter gamma is the adiabatic index for the equation of state of the gas,

$$P = K\rho^\gamma. \quad (\text{A.1})$$

Here, K is a proportionality constant and γ typically has a value of 5/3. For historical reasons, Poisson's equation for a gas of adiabatic index γ is often written in dimensionless form as the *Lane–Emden equation* (Lane 1870; Emden 1907),

$$\frac{1}{\xi^2} \frac{d}{d\xi} \left(\xi^2 \frac{d\theta}{d\xi} \right) + \theta^n = 0, \quad (\text{A.2})$$

where ξ is dimensionless radius, $\theta = \phi/\phi_0$ is the dimensionless potential, and the polytropic index $n = 1/(\gamma - 1)$. Solutions to the Lane–Emden equation result in finite-radius stars for $0 < n < 5$, while stars with $n \geq 5$ have infinite radii and stars with $n > 5$ have infinite mass. The Eddington model for radiative stars, such as the Sun, corresponds to $n = 3$. The adopted value for n in Fi in AMUSE is $n = 3/2$, or $\gamma = 5/2$, which ensures energy conservation in systems without shocks or dissipative processes. We note that hydro codes, such as Fi, contain many such parameters, each of which might be worth an extensive discussion.

Apart from the adiabatic indices, the most important free parameters in SPH include `n_smooth`, the number of neighbors over which the results are smoothed, `sph_h_const`, specifying a fixed smoothing length, if desired, and `sph_artificial_viscosity_eps`, and `sph_viscosity`, controlling the viscosity of the simulated gas. The first, `n_smooth`, is a tunable parameter, which is set to a relatively small number (64 by default). For modern high-resolution simulations, one might wish to set this to a higher value.

Some hydro codes have, in addition to the already mentioned long list of free parameters, specific time-step parameters to control drifts in velocity and acceleration. These parameters are named `free_timestep_crit_constant_a`, `..._aexp`, and similarly for the velocity with a `..._v` instead of `..._a`, for the velocity and acceleration.

Some of the flags may sound unfamiliar, such as `balsara_flag`. This parameter in the SPH code Fi refers to the activation of the Balsara switch (or Balsara limiter), a numerical technique used to reduce the effect of artificial viscosity in regions of effective shear, while still allowing artificial viscosity to be effective. Enabling the Balsara flag is important in simulations of differentially rotating disks, galaxies, or accretion flows, where shear is important and artificial viscosity otherwise dampens real structures.

In AMUSE, the Balsara limiter is turned off by default (see Table A.5), but it is one of these parameters that make the dissipation and viscosity in a disk simulation more realistic. We emphasize that default values are what they are for a reason, and changing them is risky, but we recognize that not changing them in certain circumstances may be risky too.

A.1.2.1 Tunable Radiative-transfer Parameters

A parameter not mentioned in Table A.6 is `ionization_temperature_solver`, which is really a flag for various options in solvers. The default value of 2 here indicates the backward differentiation formula (or bdf) method for solving the equations of motion (Gear 1967). It is an implicit time integration scheme, which solves the time-dependent radiative-transfer equations on SPH data, the bdf solver is used to advance the solution of the radiative-transfer equations in time, particularly when dealing with stiff equations that can arise in radiative processes. An alternative to the bdf solver would be to set `ionization_temperature_solver` to 1, in which case an explicit Euler solver is used.

The bdf solver allows for larger time steps and improved numerical stability compared to explicit Euler methods, especially when the radiative-transfer equations involve rapid changes or processes with widely varying timescales. This makes it suitable for accurately and efficiently modeling the evolution of ionization and temperature in simulations where radiative transfer is coupled to hydrodynamics.

Another common parameter in radiative transfer is `global_helium_mass_fraction`. By default, this fraction is set to 0, because in astrophysics we often deal with Hydrogen, rather than Helium-rich interstellar gas. It is one of these parameters that is important to set properly depending on your specific application.

The `isothermal_flag` controls whether to evolve temperature (0) or keep the gas isothermal (1). The `boundary_condition` flag can either be reflective (use a value of -1), vacuum (0), periodic (-1). In a more modern code, one would probably opt for strings, rather than integers for specifying such options.

Table A.7. Recognized Read/Write File Formats

Keyword	Brief Description
<code>csv</code>	Comma-separated values file
<code>dyn</code>	Starlab binary structured file
<code>starlab</code>	Extensive Starlab data format
<code>gadget</code>	Gadget binary data file
<code>hdf5</code>	HDF5 file
<code>nemo</code>	NEMO binary structured file
<code>tsf</code>	NEMO binary structured file
<code>txt</code>	Text file containing a table of values separated by a space
<code>vts</code>	Text file containing a table of values separated by a predefined character
<code>vtu</code>	Text file of unstructured XMLVTK-formatted data
<code>amuse</code>	Default selection (uses HDF5)

For a full appreciation of this routine, we refer to the source codes in `amuse.io`, in particular in `base` for the base class of the storage module, and `store` for the specifics.

A.1.3 Input/Output File Types

In Section 2.2.4 we discussed how to write and read particle sets from permanent storage using `write_set_to_file` and `read_set_from_file`. Normally we adopt `hdf5` binary formatted files for input and output, but there are circumstances in which another format might be desirable. Table A.7 presents an overview of the file types recognized in AMUSE. There are restrictions to some file formats: ASCII files, for example, will not be able to represent pointers other than by showing the address which is not very useful when reading the data from permanent storage. Of course, we could add other file formats, but we think that with this repertoire covers some of the most common ones. The default AMUSE file format is `hdf5`, which is widely accepted within the astronomical community.

Each of these storage format settings has its own derived class and associated Python file. The module `amuse.io.starlab` specifies the starlab data format, whereas the module `amuse.io.gadget` does the same for gadget. If you want to make your own derived data format, the best way to start is to copy one of the existing derived class files and edit it to your needs, filling in the specific requirements of your module.

A.2 Initial Conditions

A.2.1 Generalized Initial Condition Generating Routines

In Chapter 3, we discussed a few examples of initial conditions. Of course, any output from any high-fidelity simulation can be considered as input for another simulation. Table A.8 presents a list of the most common initial conditions generating functions and routines (as of the publication of this book). Some of these create new data, others append or manipulate data. Many of these generator functions are part of the standard AMUSE sandbox, but a large number of

Table A.8. Overview of the Routines for Generating and Manipulating Initial Conditions

Particle-set Generators	
Stars	make_single_star.py, make_binary_stars.py, make_stars_in_mass_function.py, make_cluster_of_stars.py, make_cube_of_stars.py, make_plummer_sphere.py, make_gas_plummer.py
Planets	make_solar_system_planets.py, make_exoplanetary_system.py, make_nice_like_system.py, make_planets_oligarch.py, new_solar_system_with_sun_and_planets.py.
Complexes	make_hierarchical_cluster_with_planets.py.
Append to Particle Set	
Planets	add_asteroids_to_solar_system.py, add_comets.py, add_debris_disk.py, add_moons_to_solar_system_planets.py, add_planets_oligarch.py, add_proto_oort_cloud.py, add_scattered_asteroids.py.
Stars	add_binary_companion.py, add_star_in_orbit.py.
Complexes	add_perturbation_to_model.py.
Manipulators	
Manipulators	move_all_particles.py, move_bodies_to_stellar_position.py, rotate.py.
Diagnostics	determine_orbital_elements.py, orbital_elements_to_cartesian.py.
Converters	conv_amuse_to_brutus.py, name_stars.py, write_set_as_ascii.py.
Plotting	plot_bodies.py, print_snapshot.py.

additional routines can be found on GitHub at <https://github.com/amusecode/initial-conditions-generator>.

These routines are used as follows (see also Figure A.1):

```
python make_plummer_sphere.py --nstars 16 --radius 0.01 -Q 0.3 -F Pl16.amuse
python name_stars.py -f Pl16.amuse --nstars 12 --name system -F PlN.amuse
python add_planets_oligarch.py -f PlN.amuse --fplanets 1 --name system -F PlNO.amuse
python add_debris_disk.py -f PlNO.amuse --name system --ndisk 300 -F PlNOA.amuse
python rotate.py -f PlNOA.amuse -F Pl_N16NO_A100OR.amuse --type star
```

This daunting series of calls results in a Plummer distribution of 16 stars, 12 of which have a Oligarchic planetary system, totaling 44 planets, including a debris

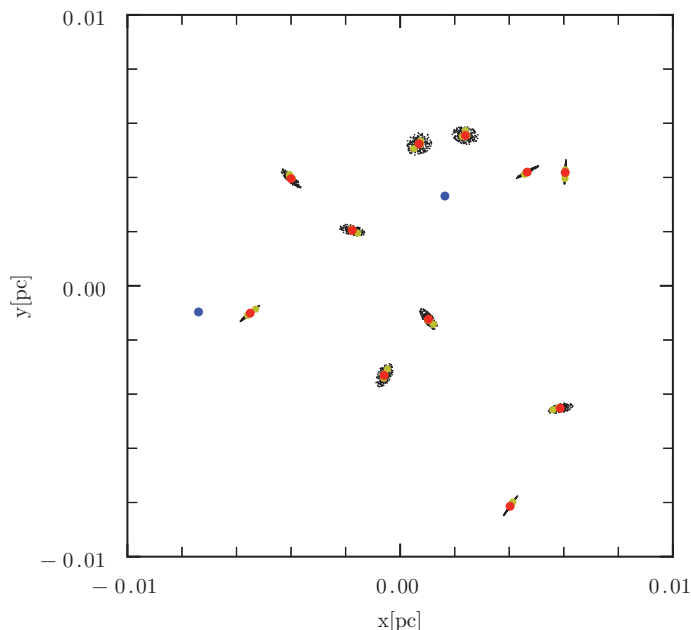


Figure A.1. Representation of a simple 16 star cluster with planetary systems and disks. Single stars are blue bullets, the host star for a planetary system is red. Planets are yellow (hardly visible), but they can be perceived since the stars are surrounded by tiny black dots (the debris). The script to generate these initial conditions is the shell script `make_cluster_with_planets.sh` (which we copied below), and the code to plot the results is `amuse.examples.plot_plummer_stars_and_planets`.

disk of 3816 test particles each in random orientations in space. The results are stored in a series of intermediate files `P116.amuse`, `P1N.amuse`, `P1NO.amuse`, `P1NOA.amuse`, and eventually the final file `P1_N16NO_A1000R.amuse` containing the initial realization of the simulation. Figure A.1 shows this cluster.

The shell script `make_cluster_with_planets.sh` generates a cluster of stars of which some stars have planets and a disk of planetesimals. The resulting data file (`P1_N16NO_A1000R.amuse`) can be plotted using the script `amuse.examples.plot_plummer_stars_and_planets`, resulting in Figure A.1.

A.2.1.1 Uniform Gas Distributions for SPH

Some codes for generating initial conditions are quite versatile, and can take numerous arguments to tune their function. The classes provided by `spherical_model` in `amuse/ext` are examples of such rich functionality. The Python class `UniformSphericalDistribution` in `amuse.ext.spherical_model` forms the basis for a number of associated routines, including `new_uniform_spherical_particle_distribution`, `new_spherical_particle_distribution`, and `new_plummer_spatial_distribution`. The underlying class generates a uniform spherical grid of particles, which can then be used for more

Table A.9. Additional Options Used in the Class `UniformSphericalDistribution` to Tune the Generation of Initial Gas-Density Distributions

Keyword	Brief Description
<code>cubic</code>	Cubic grid with particles at each corner
<code>bcc</code>	Body-centered cubic, with additional particles at the center of each cube
<code>fcc</code>	Face-centered cubic, but with additional particles on the face of each cube
<code>random</code>	Particles distributed randomly using <code>numpy.random.uniform</code>
<code>glass</code>	As for <code>random</code> , but stabilized using hydro pressure without gravity
<code>sobol</code>	3D sobol sequence (low discrepancy, quasirandom) (see Sobol 1967)

For the `sobol` method, low discrepancy indicates that the points are on average close to equally distributed.

useful initial distribution functions. Some of the most important parameters used in the class are listed in Table A.9.

To generate a spherical quasirandom distribution of $N = 10000$ particles with radius $R = 1 \text{ | units.km}$ and mass $M = 1 \text{ | units.kg}$, we write

```
from amuse.ext.spherical_model import new_uniform_spherical_particle_distribution
particles = new_uniform_spherical_particle_distribution(N, R, M, type="sobol")
```

The class has two more options: `offset` and `target_rms`. Option `offset` is only used for the regular grids (`"cubic"`, `"bcc"`, `"fcc"`) and should be specified using a tuple of three numbers in the range $[0, 1]$, which define the offset between the origin of the grid and the corner of the unit cell, normalized to the unit cell size. Option `target_rms` is only used for `glass` as the rms density criterion for convergence (see Section 9.3.2.1).

A.2.2 Writing Composite Datafiles

Simulations such as those presented in Section 9.4.4 and Chapter 10 often require writing composite files, which in Section 9.4.4 include both gravitating and gaseous particles. There is no best or optimal way to write composite data files, but we recommend keeping the format as simple as possible. In the cases just referenced, we accomplish this by creating a common particle set containing all particles and writing this to file. This particle set contains a variety of attributes, depending on the types of particles resulting from several independently operating codes, which can later be disentangled to recover the individual components.

Such a common particle set is generated by the `ParticlesSuperset` function:

```
moving_bodies = ParticlesSuperset([gravity.code.particles, hydro.gas_particles])
```

It can later be saved by

```
write_set_to_file(moving_bodies, filename)
```

or subsequently read from a file:

```
moving_bodies = read_set_from_file(filename)
```

Here, `filename` is the name of the input or output file. In this example, we write the data in the default `amuse` format (which uses `hdf5`), but several of the alternatives listed in Table A.7 could be used.

Another way to write composite data files is by simply storing the various particles sets successively, subsequently recovering them in the same order. In this example, we would create two files (using the default, we omitted specifying the data format):

```
write_set_to_file(gravity_particles, gravity_filename)
write_set_to_file(hydro_particles, hydro_filename)
```

and in reverse read both files:

```
gravity_particles = read_set_from_file(gravity_filename)
hydro_particles = read_set_write_from_file(hydro_filename)
```

A.3 Notes on Specific Codes

A.3.1 Test Particles in N -body Codes

Some codes can handle test masses, objects with zero mass. They can be quite handy for following the gravitational potential. However, if you have to pay the N^2 -price for integrating them, they do not make a lot of sense. Some of the N -body codes in AMUSE are therefore equipped to treat zero-mass objects as test masses. This feature is enabled in `Huayno` and `ph4`, and maybe more codes will be enabled with this handy feature as this book goes to print. To enable test-mass integration, simply set the mass of a particular particle to 0 (or to some negative value).

```
particle.mass = 0 | nbody_system.mass
```

Once set to zero, other nonzero-mass particles accelerate the test mass, but the test mass does not affect any other particles.

A.3.2 Post-Newtonian Integration with Relativistic Cross Terms

Between 1907 and 1915, Einstein developed general relativity (see Weinstein 2012; Poisson & Will 2014; for an interesting read on the history of this development) and viewed gravity as the result of the curvature of spacetime (Einstein & Grossmann 1914; Einstein 1915b, a; Hilbert 1915). The Einstein field equations describe the gravitational interaction between particles, but these equations are nonlinear and notoriously hard to solve. Schwarzschild (1916) found the first nontrivial solution to the Einstein field equations. The Schwarzschild metric describes a point-like particle. A rotating black hole was first described analytically as a solution to the field equations by Kerr (1963). Today, there are standard software implementations to solve general relativistic dynamical systems (Mewes et al. 2018), even including magnetic fields (Mewes et al. 2020). However, simulating multiple black holes in a relativistic context is very expensive in terms of computer time.

The *N*-body code Hermite-GRX includes relativistic effects in the post-Newtonian approximation. It was introduced in Portegies Zwart et al. (2022). Apart from the usual parameters of any direct *N*-body code, Hermite-GRX has four extra settings, listed in Table A.10.

The parameters `integrator` and `perturbation` control the integration method and the perturbation. Integrators include various forms of symmetrization and regularization. Both help in speeding up the code and reducing energy errors. The perturbation is mainly used to include various post-Newtonian effects, the first post-Newtonian and 2.5 post-Newtonian orders of the Einstein Infeld Hoffman equation, and the first post-Newtonian equations of motions that include the relativistic cross terms (useful for multiple black holes).

Table A.10. Overview of the Most Common Tunable and Nontunable Parameters for General Relativity Calculations in the Gravitational *N*-body Problem

Tunable Parameters			Gravitational Dynamics
Parameter Name	Value	Unit	Explanation
<code>integrator</code>	Hermite		Integrator type (see Table A.11)
<code>light_speed</code>	1	Length/time	Speed of light
<code>num_threads</code>	1		Number of threads on the processing core
<code>perturbation</code>	None		Perturbation mode (see Table A.11)

Table A.11. Integrators Available in Hermite-GRX

Parameter Name	Keyword
<code>integrator</code>	Hermite (default) SymmetrizedHermiteRegularizedHermite SymmetrizedRegularizedHermite
<code>perturbation</code>	None (default), 1PN_Pairwise, 1PN_EIH2.5PN_EIH

These integrators can be selected by providing the appropriate term in the mode keyword as an argument for Hermite-GRX.

In addition, Hermite-GRX has two additional particle types, `large_particles` and `small_particles`. The former is used for particles that are relativistically affected, whereas the others remain unaffected by general relativity. Normally, assigning new particles to the general particle set (`particles`) will cause all objects to be affected by general relativistic effects. But for performance considerations, sometimes one might decide to assign certain minor bodies to the `small_particles` set, excluding them from relativistic effects.

A.3.3 The Huayno Multicomponent Symplectic *N*-body Code

Most codes use specific parameters to control their internal solvers. An example of a code with a quite extensive parameter set is Huayno (Pelupessy et al. 2012), which uses a recursive Hamiltonian splitting strategy (much like `bridge`, see Chapter 9) to generate multiple-time step integrators that conserve momentum to machine precision. A wide variety of different integrators is available within Huayno. They are listed in Table A.12, but for more details we refer to Pelupessy et al. (2012). The code was developed within the AMUSE environment, and can be compiled with GPU support using OpenCL (for which the mode "OpenCL" should be specified in the run-time environment). Another gravity code with many options is Rebound (see Section A.8).

Huayno has two types of shared but variable time step codes: with and without a treatment of collisions. Both types come in all even orders, from second to 10th. Such an integrator is initialized as follows

```
gravity = Huayno(mode="SHARED10_COLLISION")
```

or equivalently by

```
gravity = Huayno()
gravity.set_integrator("SHARED10_COLLISION")
```

Table A.12. Integrators Available in Huayno

Brief Description	Keyword
Shared time step	SHARED2, ..., SHARED10,
Collision-enabled	SHARED2_COLLISIONS, ..., SHARED10_COLLISION
Kepler solvers	KEPLER, CC_KEPLER, CCC_KEPLER, OK
Connected components	CC, CCC_BS, CC_BSA, CCC_BSA
Bulirsch–Stoer	SHAREDBS, BS_CC, BS_CC_KEPLER
KDK	PASS_KDK, HOLD_KDK, BRIDGE_KDK
Drift–kick–drift	PASS_DKD, HOLD_DKD, PPASS_DKD, BRIDGE_DKD

These integrators can be selected by providing the appropriate term in the mode keyword as an argument for Huayno,

The Kepler solvers evolve the two-body problem, with a variety of alternative approaches. The OK solver (see Pelupessy et al. 2012), for example, adopts the optimal-kick Hamiltonian splitting method to evaluate every interaction in the system at the optimal-split time step, thereby using the least number of force evaluations to evolve the system.

The designation CC in Table A.12 stands for connected components. In the CC method, we split the Hamiltonian to find isolated subsystems which are integrated separately for better accuracy compared to the rest system (see also Section 10.1.2). The CC method is applicable not only to Kepler solvers but also to the Bulirsch–Stoer integrator (Bulirsch & Stoer 1964). This latter integrator can adopt shared time steps and run with connected components or with a Kepler solver. Each of these leads to better accuracy, particularly for hierarchical systems with complex configurations.

Finally, Huayno supports various types of KDK and drift–kick–drift schemes, much as in `bridge` (see Section 9.1).

A.3.4 Symple: The Simplest Possible Symplectic N -body Integrator

An example of a code that was written specifically for AMUSE is the direct symplectic N -body code `Symple`. Since there is no documentation on this code, we use it here as an example of a code that was written as an AMUSE module. `Symple` is simple, and symplectic, hence its name. The code can operate as a symplectic N -body integration module of orders 1, 2, 3, 4, 5, 6, 8, and 10, with fixed, shared time steps. Alternatively, setting the time step parameter $\eta > 0$ enables variable and shared time steps, as in, say `Hermite` (but the integrators then lose their symplectic quality). The default is $\eta = 0.05$.

Initially, we developed `Symple` to be able to effectively integrate the orbit of the triple θ Muscae (see Section 8.4.5). As we saw, this triple comprises three Wolf–Rayet stars, each with a hefty stellar wind. The combination of tight orbits, massive stars, and copious stellar winds makes this system extremely challenging to integrate. It turns out that the linear coupling strategy adopted in Section 8.4.5 is slow and prone to numerical errors. Using a `bridge` for this system makes the integration more robust, but at an enormous cost in computing time. This made us decide to write `Symple`, a simple symplectic N -body integrator that included a mass-loss term for each star.

Although symplectic integrators do conserve energy on the long run much better than the usually adopted explicit Hermite-type integrators, they can still lead to precession in an elliptic two-body orbit. Orbital phase and the argument of pericenter are not conserved quantities, and therefore are very hard to preserve in a numerical simulation. Although generally this hardly has only minor consequences for the integration, in some cases, such as when integrating von Zeipel–Lidov–Kozai (Lidov 1962; Kozai 1962) affected triples, it is of crucial importance.

Even in symplectic integrators, noncommensurate steps lead to long-period modulations in the energy error, without a long-term drift. The amplitude of the modulation can be reduced by making the step an integral fraction of the period, and reduced more by making it an integral fraction of the actual (numerical) period.

Table A.13. Symple Integration Order and Number of Evaluation Steps

Integrator/Order	Function Evals/Step	Evals/Period (Const. dE)
1	1	—
2	1	32k
3	3	3k
4	3	3k
5	7	1792
6	7	1792
8	15	3840
10	33	4224

`Symple` is not as simple as the name suggests: it facilitates an implicit Euler (Euler 1768), second-order predictor-corrector, and higher-order symplectic integrators. The choice of integrators is made in your script through the code-related parameter. In Table A.13 we present the various integration possibilities. But let us first discuss the advantages and disadvantages of each of the integrators. It is important to bear in mind that a simple symplectic integrator, like `Symple`, works best if the time step is a power of 2 times the orbital period. Not obeying this requirement leads to a rather large drift in orbital phase. For higher-order integrators (eighth- and tenth-order schemes) this time step requirement is not necessary.

The first order scheme is really an implicit Euler integrator. It has no appreciable long-term energy drift. Instead, it will exhibit large orbital variations. For longer time steps, the orbit will modulate with small amplitude. The second-order predictor-corrector scheme in `Symple`, however, is probably your best choice for most problems. Long-term orbital integration hardly shows any net drift of energy, exhibits long-term periodic modulation, and the integration is not appreciably affected by commensurate/noncommensurate steps. We recommend making the step commensurate with the numerical period, which reduces the modulation amplitude.

For the order of integrator in `Symple` to 10th order, see Table A.13. The first column gives the order of the integrator, followed by the number of evaluations per step, which quickly grows beyond sixth order. The last column gives the number of actual evolutions of the integrator per orbital period of a standard binary system (in N -body units $m1 = m2 = 0.5$, $a = 0.57$, $e = 0.75$, resulting in an orbital period of $P = 2.71$) for constant error in the energy. Considering these values, it may be hard to see why anybody would ever use a sixth-order symplectic integrator for solving a regular N -body problem, but for multiobject integrations these numbers will be different and there may be a sweet spot in parameter space for optimal use of a 10th-order symplectic integrator.

A.3.4.1 Radiative Transfer in SPH codes with SPHray

SPHray (Altay et al. 2008) is an SPH ray tracer designed to solve the three-dimensional, time-dependent, radiative-transfer equation. The method relies on a Monte Carlo ray-tracing scheme that does not interpolate the SPH particles on to a grid but instead integrates directly through the SPH kernels. An Axis-Aligned Bounding Box (AABB) (Zhao et al. 2021) test taken from computer graphics applications allows for the acceleration of the ray-tracing component. The AABB is defined as a rectangular box (in three dimensions) with edges parallel to the coordinate axes.

SPHray is capable of handling multiple ionizing sources and complex geometries, making it suitable for studying complex astronomical structures such as giant molecular clouds. Its treatments for time-dependent photoionization, recombination, heating processes, and temperatures makes it excellently suited for coupling with other applications to study time-dependent processes.

The atomic rates are stored in a number of text files for various chemical species, including Hydrogen, Helium, and Deuterium, in various degrees of ionization. SPHray also supports a wide range of ionization spectra, which can be incorporated using

```
radiative = Sphray(convert=converter, mode="openmp")
radiative.set_spectra_file("spectra/thermal1e5.cdf")
```

alternative spectra include `spectra/thermal6e4.cdf`, `spectra/power2.0.cdf`, and `spectra/power4.0.cdf`.

A.3.5 External Galactic Potential Models

In many cases, we want to simulate a self-gravitating system in some background Galactic potential. In order to accommodate such studies, we have incorporated several potential models for the Milky Way and other point-symmetric potentials.

The `galpy` (Bovy 2015, 2014) package includes a bulge modeled as a power-law density profile with an exponential cut-off, a Miyamoto & Nagai (1975) disk, and a Navarro–Frenk–White (NFW; Navarro et al. 1996) dark-matter halo. The parameters of each of the individual components are based on fits to observational data, with the additional overall constraint that the Sun’s distance to the Galactic center is $R = 8.0$ kpc and the solar circular velocity is $v = 220$ km s^{−1} Bovy (2014, 2015). We presented an example of how to initialize such a model in Section 9.4.1.1.

The asymmetric disk of Miyamoto & Nagai (1975) can also be run separately, in which case a different bulge and halo model can be added. For example, it can be complemented using the spherically symmetric Plummer (1911) profile. These profiles can be found in `amuse.ext.galactic_potentials`. The Galactic disk model is initialized using the following lines:

```

disk_mass = 1.0e+10 | units.MSun
disk_size = 10 | units.kpc
disk_height = 100 | units.parsec
from amuse.ext.galactic_potentials import MiyamotoNagai_profile
galactic_disk_model = MiyamotoNagai_profile(disk_mass, disk_size, disk_height)

```

A point-symmetric Plummer potential model can be initiated in a similar fashion using

```

total_mass = 1.0e+6 | units.MSun characteristic_size = 3 | units.parsec
from amuse.ext.galactic_potentials import Plummer_profile
globular_cluster_model = Plummer_profile(total_mass, characteristic_size)

```

In Table A.14, we present a brief overview of the semianalytic potentials that are directly available within AMUSE.

The most versatile and complete semianalytic potential model for the Milky Way is Galaxia (Allen & Santillan 1991; Cox & Gómez 2002). This parameterized galaxy-evolution model includes multiple potentials to describe the Milky Way or another galaxy. To make a representation of the current Milky Way, a number of parameters are set by default. They are listed in Table A.15.

An alternative would be to use an external package for either generating complex initial conditions based on some predefined distribution function, or use a specific background potential. The AGAMA package, discussed in Section B.2.2 introduced in Vasiliev (2019), will prove to be quite useful for such studies. It would require the separate compilation of AGAMA, though, which is explained in Section B.2.2.

Table A.14. Short Description of the Potential Models Available in AMUSE

Model	Brief Description
NFW_profile	Navarro, Frenk, & White (Navarro et al. 1996) dark matter profile. Free parameters include the scale radius and central density.
MiyamotoNagai_profile	Semianalytic disk model (Miyamoto & Nagai 1975). Free parameters include the total mass and the disk scale size and height.
Plummer_profile	Plummer density profile (Plummer 1911). Free parameters include the total mass and the scale radius.
PowerLawCutoff_profile	Spherically symmetric power-law density profile with exponential cut-off. Free parameters include the central density, power-law scaling radius, the power-law index and the cut-off radius.
MWpotentialBovy2015	Jo Bovy’s galpy density profile model (Bovy 2015).
Galaxia	Galaxy potential with bulge, bar, disk, spiral arms and halo (Sharma et al. 2011).

Table A.15. Default Parameters for the Milky Way in `galaxia` (See `amuse.examples.solar_cluster_in_galaxy_potential` discussed in in Section 9.4.1.1)

Parameter Name	Value	Unit
<code>phase</code>	-0.34906	...
<code>a_disk</code>	5.3178	kpc
<code>a_halo</code>	12.0	kpc
<code>aaxis_bar</code>	3.12	kpc
<code>amplitude</code>	1300	$\text{kms}^2 \text{kpc}^{-1}$
<code>amplitude2</code>	680	$\text{kms}^2 \text{kpc}^{-1}$
<code>axis_ratio_bar</code>	0.37	...
<code>b_bulge</code>	0.3873	kpc
<code>b_disk</code>	0.25	kpc
<code>bar_phase</code>	-0.349	...
<code>caxis_bar</code>	1.0×10^{-6}	kpc
<code>fiducial_radius</code>	8.0	kpc
<code>initial_phase</code>	-0.34906	...
<code>mass_bar</code>	1.4000×10^{10}	$M_{\odot}$
<code>mass_bulge</code>	1.4059×10^{10}	$M_{\odot}$
<code>mass_disk</code>	8.5608×10^{10}	$M_{\odot}$
<code>mass_halo</code>	10.7068×10^{10}	$M_{\odot}$
<code>nbt</code>	0.0	...
<code>omega_bar</code>	40	km (s kpc)^{-1}
<code>omega_spiral</code>	20	kms (kpc)^{-1}
<code>omega_spiral2</code>	20	km (s kpc)^{-1}
<code>omega_system</code>	40	km (s kpc)^{-1}
<code>phi2l_spiral</code>	-3.49	...
<code>r_sigma</code>	2.5	kpc
<code>rsp</code>	3.12	kpc
<code>scale_height</code>	0.18	kpc
<code>sigma_s</code>	100	Myr
<code>spiral_density_amplitude</code>	5.80×10^{14}	$M_{\odot} (\text{kpc})^{-3}$
<code>spiral_model</code>	0.0	...
<code>spiral_phase</code>	-0.349	...
<code>t_sim</code>	-34.132	Myr
<code>tan_pitch_angle</code>	0.227	...

These are many free parameters in this model. Here, we have rounded off some of the numbers; in the package, the pitch angle is given to 9 decimal places and the scale height of the disk to 12 digits.

You can call `Galaxia` and set the mass of the disk to $1.0 \times 10^{11} M_{\odot}$ as follows:

```
from amuse.community.galaxia.interface import BarAndSpirals3D
galaxy = BarAndSpirals3D()
galaxy.parameters.mass_disk = 1.0e+11 | units.MSun
```

All the parameters listed in Table A.15 can be set separately to construct original background-potential models.

A.4 Higher-order Bridges

The standard bridge is second order, and here we derive the fundamental method. As a result, the numerical error (in particular in energy) scales with the square of the time step dt^2 . Sometimes, higher-order integration methods are desirable. For this reason, we present some higher-order bridge schemes.

A.4.1 Fundamentals of High-order Bridges

In some cases, the Bridge scheme may encounter numerical difficulties because the method is only second-order accurate in the time step dt . This can become a limiting factor when coupling two higher-order methods (e.g., sixth- and fourth-order Hermite schemes), or when coupling a fourth-order scheme with a background potential expressed as an analytic function or look-up table. For these cases, a higher-order bridging method is desirable, especially if the symplectic quality can be preserved. Here, we show how Bridge can be extended to an arbitrarily high (even) order. There exist symplectic integrators of odd order, for example the implicit Euler, but they are not very suitable for constructing bridges (Jiao et al. 2007).

We can derive higher-order bridges formally as follows. This starts by assuming a system of particles composed of some number Q of subsystems k . In this case, the total Hamiltonian of the system,

$$H = \sum_{i \in S} \frac{\mathbf{p}_i^2}{2m_i} - \sum_{\substack{i, j \in S \\ i \neq j}} \frac{Gm_i m_j}{r_{ij}}, \quad (\text{A.3})$$

can be split to obtain

$$H = \sum_{k=1}^Q H_k + \sum_{\substack{k, l=1 \\ k \neq l}}^Q H_{kl}^{int}. \quad (\text{A.4})$$

The terms in Equation A.4 are given by

$$H_k = \sum_{i \in k} \frac{\mathbf{p}_i^2}{2m_i} - \sum_{\substack{i, j \in k \\ i \neq j}} \frac{Gm_i m_j}{r_{ij}} \quad (\text{A.5})$$

$$H_{kl}^{int} = - \sum_{\substack{i \in k \\ j \in l}} \frac{Gm_i m_j}{r_{ij}}.$$

Note that this is a slightly different notation than in the original paper by Fujii et al. (2007), but the end result is the same.

Based on this splitting, a general, multisubsystem, second-order time evolution operator can be constructed as follows:

$$\text{Bridge}_2(\tau) = \prod_{k=1}^Q e^{\frac{1}{2}\tau D_{H_k}} \prod_{\substack{k,l=1 \\ k \neq l}}^Q e^{\tau D_{H_{kl}^{\text{int}}}} \prod_{k=1}^Q e^{\frac{1}{2}\tau D_{H_k}}. \quad (\text{A.6})$$

As in the classic Bridge, the operators $e^{\tau D_{H_k}}$ independently evolve each of the subsystems k in isolation. The operators $e^{\tau D_{H_{kl}^{\text{int}}}}$ represent pure momentum kicks due to interactions between subsystems k and l . In the case of the Hamiltonian in Equation A.4, the interaction terms do not depend on velocity, so the operators $e^{\tau D_{H_{kl}^{\text{int}}}}$ commute. We note, however, that this is not the case for velocity-dependent forces, so special treatment is required in that case. Because each of the operators $e^{\tau D_{H_k}}$ and $e^{\tau D_{H_{kl}^{\text{int}}}}$ can, in principle, be associated with different solvers running concurrently, a highly efficient parallel implementation can be achieved for the generalized Bridge scheme.

A higher-order symplectic Bridge scheme can be constructed in a similar fashion. By defining the drift and kick operators

$$\mathcal{D}(\tau) = \prod_{k=1}^Q e^{\tau D_{H_k}} \quad (\text{A.7})$$

and

$$\mathcal{K}(\tau) = \prod_{\substack{k,l=1 \\ k \neq l}}^Q e^{\tau D_{H_{kl}^{\text{int}}}}, \quad (\text{A.8})$$

Equation A.6 can be extended to higher order by a suitable combination of $\mathcal{D}(\tau)$ and $\mathcal{K}(\tau)$ operators (Hairer et al. 2006). For a fourth-order symmetric composition with four stages, the high-order Bridge takes the form:

$$\begin{aligned} \text{Bridge}_4(\tau) = & \mathcal{D}(u_0\tau) \mathcal{K}(u_0\tau) \mathcal{D}(u_1\tau) \mathcal{K}(v_1\tau) \\ & \mathcal{D}(u_2\tau) \mathcal{K}(v_1\tau) \mathcal{D}(u_1\tau) \mathcal{K}(v_0\tau) \mathcal{D}(u_0\tau) \end{aligned} \quad (\text{A.9})$$

(see Yoshida 1990; Hairer et al. 2006; for the coefficients u_i and v_i). The sixth-order method becomes

$$\begin{aligned} \text{Bridge}_6(\tau) = & \mathcal{D}\left[\frac{1}{2}w_0\tau\right] \mathcal{K} \\ & [w_0\tau] \mathcal{D}\left[\frac{1}{2}(w_0 + w_1)\tau\right] \cdots \mathcal{D}\left[\frac{1}{2}(w_{s-1} + w_s)\tau\right] \mathcal{K}[w_s\tau] \mathcal{D}\left[\frac{1}{2}w_s\tau\right], \end{aligned} \quad (\text{A.10})$$

where the coefficients w_i are given by Sofroniou & Spaletta (2005) and Hairer et al. (2006). The higher-order bridges are identified by order: Leapfrog, fourth, sixth, etc., and by the number of force evaluations: $m = 5$ or $m = 6$ for the fourth order, $m = 11$ or $m = 13$ for the sixth order. AMUSE includes implementations up to and including 10th order (with $m = 35$), which you will probably never need. A complete list of methods for bridge can be found in Section A.4.2 (see Table A.16).

Table A.16. Bridge Methods Identified by the Order of Their Integrator

Name	Order	S	M
LEAPFROG	...	...	...
SPLIT_4TH_S_M4	4	S	4
SPLIT_4TH_S_M5	4	S	5
SPLIT_4TH_S_M6	4	S	6
SPLIT_6TH_SS_M11	6	SS	11
SPLIT_6TH_SS_M13	6	SS	13
SPLIT_8TH_SS_M21	8	SS	21
SPLIT_10TH_SS_M35	10	SS	35

The columns give the order of the integrator, the splitting strategy (S for symmetric composition and SS for symmetric composition of symmetric maps), and the number of evaluations per bridge step (M).

The above formulation provides a fully symplectic time evolution if the codes being bridged are themselves symplectic. Using this scheme, codes/integrators of different orders can be coupled to construct a formally high-order scheme by matching the order of the bridge used during the coupling. For example, when coupling a sixth order to a fourth-order method, it is appropriate to choose $\text{Bridge}_4(\tau)$ to create a fourth-order compound method. If we chose $\text{Bridge}_6(\tau)$, the method would still be only fourth-order because of the fourth-order subintegrator. In what follows, we refer to the second-order (original) bridge as the classic bridge, and use the term higher-order bridge for the generic case.

The logic of selecting an appropriate bridge order may not always be straightforward. For example, if second-, fourth-, and sixth-order schemes were coupled using the classic bridge [$\text{Bridge}_2(\tau)$], the result would be a second-order compound method. However, we could improve this by adopting a *hierarchical coupling strategy*: bridge the fourth- and sixth-order codes using $\text{Bridge}_4(\tau)$, then bridge this compound method with the remaining second-order code using $\text{Bridge}_2(\tau)$. Although the overall compound solver is still formally second order, the subsystems evolved with the more precise codes are coupled at a higher order than the rest of the system. This can be advantageous if such a subsystem dominates the overall error or requires higher precision—for example, because of the requirement to resolve close encounters. Finally, we note that, while the methods presented here are very flexible, the proper choice of integrators is highly problem-dependent.

A.4.2 Tuning the Higher-Order Bridge

In Chapter 9 (see Section 9.3.1) we discussed coupling codes using `bridge`, and in Section A.4 we explained how higher-order bridges can be constructed by repeated steps with a bridge operator. Several higher-order bridge methods are incorporated into AMUSE; some of them worked out in `composition_methods`, which can be incorporated in your script via


```
from amuse.ext.composition_methods import {method_name}
```

where `method_name` is the name of the method you want to use. Table [A.16](#) presents a brief overview of the available methods.

An example of the working of a higher-order bridge can be found in the code example `amuse.examples.three_body_bridge_order4m4`, which we discuss in Section [9.3.1](#). There we declare a fourth-order bridge with four evaluations per bridge step, using

```
gravity = bridge.Bridge(use_threading=False, method=SPLIT_4TH_S_M4)
```

A.5 Stopping and Boundary Conditions

Any code has a domain of operation. This limitation can be controlled in terms of stopping conditions (at what point should a code hand back control to the parent script) and boundary conditions (when should a code hand back data that does not belong to its domain).

A.5.1 Hydrodynamical Boundary Conditions

In many circumstances, smoothed-particle hydrodynamics solvers do not have to worry much about boundary conditions. Simulations of star formation or stellar collisions, for example, simply remove SPH particles that stray too far from the region of interest. In other circumstances, however, boundary conditions are important, and these are implemented and accessible through the AMUSE interface (see Table [6.2](#)). For example, periodic boundary conditions in an SPH code can be declared by setting the parameter `periodic_boundaries_flag` to `True` and giving a dimension to `periodic_box_size`.

On the other hand, grid-based hydrodynamics codes, as discussed in Section [6.2.3](#), always require some sort of boundary condition, and the number of options is more extensive. For these codes, the boundary conditions are an attribute of the code that can be set at run time. For example, to apply periodic boundary conditions along the Cartesian coordinates, you could write:

```
hydro_code.parameters.x_boundary_conditions = ("periodic", "periodic")
hydro_code.parameters.y_boundary_conditions = ("periodic", "periodic")
hydro_code.parameters.z_boundary_conditions = ("periodic", "periodic")
```

Alternatives to periodic boundary conditions are `"cont," "inflow," "interface," "outflow,"` and `"reflective,"` although not all of these may be defined for each of the available hydrodynamics solvers. These terms may sound a bit arcane, but if you perform hydrodynamical simulations, they are probably quite familiar.

A.5.2 Stopping Conditions

Throughout this book we have stressed the importance of being able to stop a code cleanly when certain conditions are met. These conditions are called stopping conditions. We currently maintain a finite set of such conditions; but if more are needed, we encourage you to implement them because they make the interaction between the modules in AMUSE more flexible and versatile.

Table A.17 lists the most commonly used stopping conditions in the AMUSE community modules. Not all codes support all of the stopping conditions for that particular domain. The main reasons that some codes do not support some or all possible stopping conditions are either that the stopping conditions were never implemented (or tested, and therefore not advertised) or that a particular code does not behave well under externally initiated interrupts. Adding a new stopping condition is not hard for most codes, but it does require modifying the source code and it tends to adversely affect performance.

Table A.17. Stopping Condition Keywords for Gravitational Dynamics, Stellar Evolution, and Hydrodynamics

Stopping Condition Keywords	
Gravitational Dynamics	
<code>collision_detection</code>	Detect collision between two particles
<code>stopping_conditions_timeout</code>	Detect if the predetermined end time is reached
<code>out_of_box_detection</code>	Check if particles have left some predefined box size
<code>number_of_steps_detection</code>	Stop after predetermined number of integration steps are realized
<code>escape_detection</code>	Check if a particles has escaped
<code>pair_detection</code>	Check if two particles have become mutually bound
Stellar Evolution	
<code>supernova_detection</code>	Check if a supernova has occurred
<code>max_age_stop_condition</code>	Check of some maximum age has been reached
<code>max_iter_stop_condition</code>	Check for maximum number of internal code iteration-steps
<code>min_timestep_stop_condition</code>	Guarantee the code to stop when the time step becomes too small
Hydrodynamics	
<code>minimum_density_parameter</code>	Check for a minimum density threshold
<code>maximum_density_parameter</code>	Check for a maximum density threshold

<code>minimum_internal_energy_parameter</code>	Check for the minimum energy of a particle
<code>maximum_internal_energy_parameter</code>	Check for the maximum energy of a particle
<code>number_of_steps_parameter</code>	Parameter to set the maximum number of steps
<code>out_of_box_parameter</code>	Parameter to set the size of the box
<code>out_of_box_use_center_of_mass_parameter</code>	Assure that the out-of-box detection is done from the center of mass
<code>timeout_parameter</code>	Set the maximum simulation time

There are no dedicated stopping conditions for radiative-transfer codes, simply because up to now they have not been needed. The lack of stopping conditions for radiative-transfer codes does not mean that these are not needed, but rather that these codes have not been used as frequently yet in production simulations.

A.6 Other Handy Routines

There are quite a few handy routines in AMUSE. If ever you find yourself in a position to write a new routine because you want to manipulate a specific particle set, chances are that we already wrote such a routine. This book, however, is too short to list all possibilities and refer explicitly to each of the handy routines we have written and tested. If it exists, it is likely that our routine also turns out to be effectively optimized. On the other hand, there are many ways in which one can manipulate specific data. The best place to look for our own implementation, and hopefully in the near future also including your contributions, is in the actual source code. Particularly handy routines to handle data can be found in the following directories: `coupledata`, `datamodel`, `ext`, `ic`, `io`, `plot`, `rfl`, `support`, `test`, and `units`.

An example of such a handy routine is the method to find a subset of particles that belong to a larger particle set:

```
L4_trojans = planetesimals.get_intersecting_subset_in(L4_trojans_rot)
```

here the particle set `planetesimals` is a set that supposedly contains members of another particle set, called `asteroids`. This routine returns the particular particles in the particle set `planetesimals` that are identical to those in the other particle set `asteroids`.

It may seem that this is a rather contrived operation, but in practice we need this more often than one might naively expect. There is a total of 5000 routines in the collective AMUSE directories. Some are more useful than others, but usually there are good reasons why we have written them. Before you start writing your own data-handling routine, we strongly recommend that you first search for an existing routine that does the job.

A.6.1 Particles and Particle Sets

Particles form a fundamental part of the AMUSE framework. They are a form of containers with attributes, member functions, manipulators, and communicators. They form the heart of the AMUSE data representation.

A.6.2 Particle Attributes

Each domain has a standard selection of attributes. Some codes have additional sets. The main attributes are listed in `amuse.datamodel.attributes`.

In Tables A.18 to A.21, we list the most fundamental attributes and their default values for the various particles sets for that specific domain.

Table A.18. Overview of the Minimum Set of Attributes for the Gravity Domain in AMUSE

Attribute Name			Gravitational Dynamics Explanation
Attribute	Value	Unit	
Mass	0.0	kg	Particle mass
Radius	0.0	m	Particle radius
Position	[0.0, 0.0, 0.0]	m	Position vector of a particle
Velocity	[0.0, 0.0, 0.0]	m s ⁻¹	Velocity vector of a particle
Acceleration	[0.0, 0.0, 0.0]	m s ⁻²	Acceleration vector

Table A.19. Overview of the Minimum Set of Attributes for the Stellar Evolution Domain in AMUSE

Attribute Name			Stellar Evolution Explanation
Attribute	Value	Unit	
Mass	0.0	kg	Particle mass
Radius	0.0	m	Particle radius
Age	0.0	s	Particle age

Table A.20. Overview of the Minimum Set of Attributes for the Hydrodynamics Domain in AMUSE

Attribute Name			Hydrodynamics Explanation
Attribute	Value	Unit	
pressure	0.0	Pa	The pressure in a region of space
density	0.0	kg m ⁻³	The density of molecules or solid matter

temperature	0.0	K	The temperature of the gas
magnetic_field	0.0	T	Magnetic field created by gas and stars
velocity_field	0.0	m s^{-1}	Velocity of the gas
gravitational_potential	0.0	$\text{m}^2 \text{s}^{-2}$	Gravity forces from stars and gas
viscosity	0.0	$\text{kg m}^{-1} \text{s}^{-1}$	Viscosity of the gas cloud

Table A.21. Overview of the Minimum Set of Attributes for the Radiative-transfer Domain in AMUSE

Attribute Name			Radiative Transfer Explanation
Attribute	Value	Unit	
temperature_gas	0.0	K	The temperature of the gas
temperature_dust	0.0	K	The temperature of the dust
temperature_background	0.0	K	The temperature of the background
density	0.0	mol m^{-3}	Molecular density
magnetic_field	0.0	tesla	Magnetic field created by gas and stars
velocity_field	0.0	m s^{-1}	Velocity of the gas

A.6.3 Particle Queries and Functions

Apart from a simple storage container, there are numerous functions associated with particles and particle sets. Here, we list the most useful ones. Many of these have already been presented in various forms, but we will give a brief overview of a few of the most useful. Note that we have both singular (`Particle()`) and plural `Particles()`. The latter, we call a set. The difference is similar to an integer and an array of integers. Some functions will work on a single particle, others on a set of particles. Converting a single particle to a set can be realized using `as_set()` (see below):

- `get_timeline_of_attribute_as_vector()` returns an array of a specific attribute from the time-line of a particle set.
- `copy_to_new_particles` copies all of a particle's attribute data to a new particle.
- `new_channel_to(self, other, attributes=None, target_names=None)` makes a channel between two particle sets.
- `copy_values_of_attributes_to(self, attribute_names, particles)` copies the attribute values of one particle to another particle.
- `add_particles(self, particles)` adds a particle to a particle set.

- `ensure_presence_of(self, x)` ensures that a particle has a specific attribute.
- `remove_particles(self, particles)` removes an existing particle.
- `synchronize_to(self, other_particles)` synchronizes one particle set to another. This routine does not copy data, but makes sure that particles missing in one set will be synchronized to the other, or that particles not present in the one set are removed from the other.
- `as_set(self)` converts a single particle to a particle set.
- `select_array(self, selection_function, attributes=())` returns a subset view of a particle set, containing all particles for which the selection function returns True.
- `difference(self, other)` returns a new subset containing the difference between two particle sets.

```
particles = Particles(3)
particles.x = [1.0, 2.0, 3.0] | units.pc
subset = particles.select(lambda pos : pos < 2.5 | units.pc,
["x"])
inner_stars = particles.difference(subset)
print(inner_stars)
```

- `has_duplicates(self)` returns True when a set contains a particle with the same key more than once. Particles with the same key are interpreted as the same particle. This small example:

```
particles = Particles()
p1 = particles.add_particle(Particle(1))
p2 = particles.add_particle(Particle(2))
particles.has_duplicates()
```

returns False.

- `reversed(self)` returns a subset with the same particles, but with reversed sequential order (the first particle will become last)
- `sorted_by_attributes(self, *attributes)` sorts particles by a specific attribute. The last attribute name in the call is used for the primary sort order, the second-to-last attribute name for the secondary sort order, and so forth.

```
particles = Particles(4)
particles.mass = [2.0, 3.0, 1.0, 4.0] | units.kg
particles.radius = [3.0, 2.0, 1.0, 2.0] | units.m
sorted = particles.sorted_by_attributes('mass', 'radius')
print(sorted.radius)
```

- `random_sample(self, number_of_particles)` randomly selects a subset of `number_of_particles` particles from a parent set.

- `remove_attribute_from_store(self, name)` removes a specific attribute from a particle.

```
particles = Particles(3)
particles.temporary_attribute = 42
particles.remove_attribute_from_store("temporary_attribute")
```

Particles can be organized in supersets using the `ParticlesSuperset` particle set, which is a derived class from the `AbstractParticleSet`. In a superset of particles, attribute values are not stored by the superset, but kept locally in the subset. The superset provides a view of two or more sets of particles.

Superset objects are not supposed to be created directly, but are always derived from two or more particle sets. A simple example is

```
p1 = Particles(3)
p1.mass = [10.0, 20.0, 30.0] | units.kg
p2 = Particles(3)
p2.mass = [40.0, 50.0, 60.0] | units.kg
p = ParticlesSuperset([p1, p2])
print(len(p))
6
print(p.mass)
[10.0, 20.0, 30.0, 40.0, 50.0, 60.0] kg
p[4].mass = 70 | units.kg
print(p.mass)
[10.0, 20.0, 30.0, 40.0, 70.0, 60.0] kg
p2[1].mass
quantity<70.0 kg>
cp = p.copy()
print(len(cp))
6
print(cp.mass)
[10.0, 20.0, 30.0, 40.0, 70.0, 60.0] kg
```

A.6.4 Grid-based Data

Similarly to `Particle` and `Particles` sets, AMUSE recognizes grids. The grid class is based on the `AbstractGrid` class. This abstract class has an endless range of member functions for grid manipulation. It is best to look in the module `amuse.datamodel.grids`. Derived classes from the `AbstractGrid` include `UnstructuredGrid`, `StructuredGrid`, `RectilinearGrid`, `RegularGrid`, and `CartesianGrid`. The functions to generate those grids include `new_cartesian_grid()`, `new_regular_grid()`, `new_rectilinear_grid()`, `new_structured_grid()`,

and `new_unstructured_grid()`. And then we did not even mention the entire repository of `SubGrid` structures.

Apart from those grid-generating functions, there are numerous grid manipulation routines, query functions, attribute setters, and methods to sample the grid, similarly to the enormous range of functions associated for particles and particle sets. The number of routines is so large that even for this appendix it would go too far to mention them all. The best advice we have is to read the source code in the `amuse.datamodel` module.

A.7 Plotting

Making figures and plotting in general is an art. A good figure has a message, and it is important to have the right tools to communicate it. Here, we discuss a few Python tools that we like and have used to make the figures in this book.

A.7.1 Basic Plotting in Python

The most basic plotting can be done with `gnuplot` (Williams & Kelley 2013; Vaught 1996), which can be downloaded from <http://www.gnuplot.info>. If you have installed `gnuplot` on your computer, you can use the `gnuplot-py` package to run it directly from your script. To install `gnuplot-py`, open a shell and type

```
>% easy_install gnuplot-py
```

An extensive manual for `gnuplot` is available at: http://www.gnuplot.info/docs_5.0/gnuplot.pdf.

Sometimes more control is needed to create a plot exactly in accord with your demanding co-author's wishes. For more elaborate plotting, we have opted for `pyplot`, which is part of the `matplotlib` package (Hunter 2007). `Matplotlib` is a Python plotting library which can produce publication-quality figures. Many of the AMUSE example scripts use this library to produce graphical output, and we refer you to `amuse.examples` for many examples, from simple to complex. If you would like to take advantage of `matplotlib`, get it from <https://matplotlib.org> and install it as a Python package. For your own work, it is of course also possible to print the required data to a file and use your favorite plotting tool to make the figures.

A.7.2 Plotting with Units

Most of the figures in this book are made using `matplotlib`. In most cases, we used the standard `matplotlib` and set the units by hand. However, this is not really necessary in most cases because we have overloaded the most important functions to allow them to handle arrays with units. The AMUSE enabled `matplotlib` functions can be imported via


```
from amuse.plot import scatter, plot, hist
```

The majority of the `matplotlib` routines can be imported in this way, but we have also created a few new ones that are specific to particular codes used in AMUSE. They are listed below in Section A.7.3.

An interesting tool for plotting that supports AMUSE particle sets is `vaex` at <https://vaex.io/> (Breddels 2017). This package is particularly interesting if the particle sets become very large and hard to handle because `vaex` can process a billion objects per second on a regular workstation. This enables direct exploration of large data sets, making slices and selections using Boolean expressions. `Vaex` is easily installed from the git server using

```
%> pip install git+https://github.com/vaexio/vaex/
```

It is also possible to use the `jupyter` (Kluyver et al. 2016) version of `vaex`, which is called `ipyvolume`. This enables you to construct a portable html compliant version of your plotted data which can easily be incorporated in a web interface. This package can be downloaded from the `github` repository.¹

A.7.3 Additional Plotting Utilities

We have created supporting routines for a variety of plotting applications. Some of these routines have found their way into AMUSE. Particle-based plots are generally rather easy to make, including animations, as we demonstrated in Chapter 4. Plotting results from radiative-transfer and hydrodynamical simulations are more complicated, and for these purposes we provide a few additional plotting routines. Most can be found in the `amuse.plot` module.

- The results from SPH simulations are often easiest to interpret by simply plotting the individual particles. For this purpose, we provide the function

```
sph_particles_plot()
```

which is a very simple and fast way to make a plot of the hydrodynamical state of a set of SPH particles. It takes a particle set as argument, with several additional arguments to control the energy range, the size of the viewport, and the transparency of the particles. The particles must have `position`, internal energy `e`, and a smoothing length `h_smooth`, otherwise the routine will not know how to calculate the density maps. Figure A.2 shows the 2010 Leiden Observatory Christmas card that we made using this routine.

- For more flexible plotting, we can convert a particle set in AMUSE to a `pynbody` data set by calling²

¹ See <https://github.com/widgetti/ipyvolume>

² A concise manual for `pynbody` is available online at <https://pynbody.github.io/pynbody/>

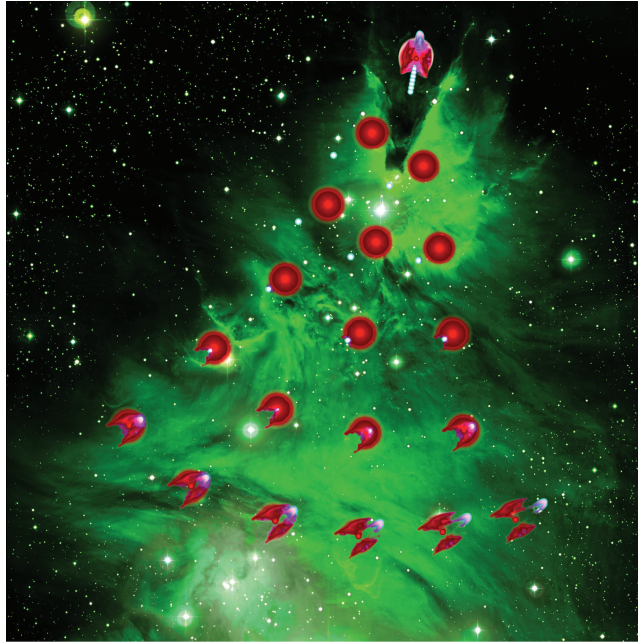


Figure A.2. False color image of the HII region NGC 2264 in the constellation Monoceros (the Unicorn). Overplotted are snapshots of a hydrodynamical simulation (using `Fi`) of two stars of masses 0.3 and $3.0 M_{\odot}$ that collide head-on with a relative velocity of 3000 km s^{-1} at an age of 310 Myr , when the primary is a giant. The Christmas card was made with AMUSE and used by Leiden Observatory in 2010. The figure was made using the script `amuse.examples.christmas_card_2010`; it takes a couple of hours to run on a laptop.

```
convert_particles_to_pynbody_data()
```

Here, in addition to providing a particle set, the arguments `length_unit` and `pynbody_unit` should be set to the desired units, for example `length_unit=units.kpc` and `pynbody_unit="kpc"`.

- Since `pynbody` is a convenient package, we also provide a column density plotting routing for it:

```
pynbody_column_density_plot()
```

There are a number of limitations to these plotting packages, and they are not as well developed as the rest of AMUSE, but they may at least serve as a basis for more complete routines.

- Finally, for plotting equipotential surfaces we provide

```
effective_iso_potential_plot(gravity_code, ...)
```

which has a large number of possible arguments. This routine creates a contour plot of the effective potential of particles in a gravity code. The code must support `get_potential_at_point`, which is used to map the iso-contour. The routine is

practical because it is almost completely independent of the underlying code, and, as a result, it is possible to make iso-density plots or equipotential-surface plots from any potential, even if the underlying potential is provided by a complicated bridging of multiple codes. It requests the gravity code from which the data are read, and some additional parameters, each of which is listed in the original source code, which can be found in the `amuse.plot` module.

A.7.4 Multiprocessing Codes

We can implement multiprocessing in this example by slightly rewriting the main evolve function in Code example 5.4. In Code example A.1, we adopt a similar approach as in Code example 5.4, but we can enable multiprocessing by introducing the specific stellar evolution code as an argument. This routine takes `code` as an argument and uses it to evolve a $10 M_{\odot}$ star to an age of 1 Myr. We could expand its functionality by introducing more arguments, such as mass, end time, and metallicity, but that would make the code example longer and not necessarily easier to understand.

The driving sequence for the main evolution routine in Code example A.1 is presented in Code example A.2. Here, we use the Python packages `queue` and `threading` for synchronizing queues and enabling threading, respectively, along with the process-based threading interface `multiprocessing`. We recognize that this example uses several fairly sophisticated multiprocessing concepts that the novice reader may find intimidating. If that is the case, you can safely skip to the next subsection.

We construct the task queue as a global variable `code_queue`, which in our case contains the list of codes used in our experiment. The actual work is done in the function `remote_worker_code`, which reads the next task from the queue, calls the evolution function `evolve_single_star`, and notifies the queue when the task has finished. The routine `evolve_with_different_stellar_model` forms the heart of the multiprocessing task. It fills the queue with the various codes, checks how many

```

58 def evolve_single_star(code):
59     stars = Particles(mass=10 | units.MSun)
60     stellar = code()
61     stellar.particles.add_particles(stars)
62     channel = stellar.particles.new_channel_to(stars)
63
64     stellar.evolve_model(1 | units.Myr)
65     channel.copy()
66     print(
67         f"Star evolved to time= {stellar.model_time} "
68         f"M= {stars.mass} R= {stars.radius}"
69     )
70     stellar.stop()

```

Code example A.1: Rewritten routine (from Code example 5.4) for evolving a single $10 M_{\odot}$ star to an age of 1 Myr with the stellar evolution code as an argument. Code fragment from `amuse.examples.multiple_stellar_threaded`.

```

33 code_queue = queue.Queue()
34
35
36 def remote_worker_code():
37     code = code_queue.get()
38     evolve_single_star(code)
39     code_queue.task_done()
40
41
42 def evolve_with_different_stellar_model(codes):
43     for code in codes:
44         if code is not None:
45             code_queue.put(code)
46     n_cpu = multiprocessing.cpu_count()
47     for i in range(n_cpu):
48         thread = threading.Thread(target=remote_worker_code)
49         thread.daemon = True
50         thread.start()
51     code_queue.join() # block until all tasks are done

```

Code example A.2.: Multiprocess version of the main loop for evolving stars, again from `amuse.examples.multiple_stellar_threaded`.

cores are available (to prevent overloading your system), and starts the threads. It ends by blocking each thread until all items in the queue have finished. The entire routine to launch multiple stellar evolution codes is given in `amuse.examples.stellar.multiple_stellar_threaded`.

Now, we can evolve the same (10 $M_{\odot}$) star to the same age (1 Myr), using four different stellar evolution codes in parallel. These codes are listed in an array and passed as an argument to the driving routine:

```

codes = [SeBa, SSE, MESA, EVTwin]
evolve_with_different_stellar_model(codes)

```

Note that simulating the evolution of a large number of stars may run into fundamental limitations of the operating system. Modern operating systems are limited, by their standard settings, to $2^{15} = 32,768$ concurrent jobs (even on large supercomputers), and it often requires root access to change this.

A.8 AMUSE Community Modules

AMUSE contains a collection of some 50 different community modules, each of which can be used separately, or in conjunction with one or more other codes. Not all combinations make sense, even though technically the codes can be combined. Here, we give a brief overview of all current community modules. This list is bound to change with time.³

³For updates, see `AMUSE_updates.pdf`.

We recognize three classes of modules, the core selection, envelope, and the rest. Originally, we designed AMUSE following a “Noah’s Ark” philosophy (Portegies Zwart 2018). According to this paradigm, each domain should have two working implementations. In practice, we sometimes deviate from this, but two modules are the minimum necessary to be able to develop a generalized interface.

Table A.22 presents an overview of the core collection of codes.

A.8.1 Modules by Domain and Functionality

Table A.23 lists the modules in AMUSE by domain and functionality. Note that, in the gravitational domain, we have expanded the code type category to include “dedicated” codes, which simply solve Kepler’s equations, and “limited” codes, which make such rigorous assumptions about the gravitational force that they should be considered potential solvers, rather than N -body codes.

Because the efficiency of a code depends on the algorithm, time-stepping, data transport, initial conditions, available hardware, parameter settings, etc., it is hard to give an absolute ranking, and in addition speed and efficiency are not always the same thing. Generally, but not always, slower codes tend to produce more accurate results, and therefore should not necessarily be avoided; the difference in speed may translate to higher accuracy or a better precision (see Portegies Zwart & Boekholt 2018 for a discussion of the difference between accuracy and precision).

A.8.2 Modules by Name

Finally, we list all current modules alphabetically, for reference. For each code, we (1) provide an indication of its main working domain, (2) point to a script in which the code’s usage is demonstrated, (3) list extra requirements or features regarding software or hardware, and give a short list of (4) equivalent and (5) comparable codes. Equivalent codes use the same or a similar algorithm to solve the same underlying equations, while comparable codes solve the same physics but using a different numerical method. Here, *the same* and *different* are used rather loosely; the terms are meant to indicate what alternative code might be used to address a particular problem. Specifically, it should be possible to substitute an equivalent code into a script designed for any given code, with minimal changes, so if a code has no script listed, try one of its equivalents.

Core selection: These are codes we support and continue to support. We either update the interface, or incorporate bug fixes or even new version of the code if they become available.

Athena (core)

Grid-based astrophysical hydrodynamics solver with support for magnetic fields (Stone et al. 2010). It was originally developed for studying the interstellar medium, star formation, and accretion flows, but can address many other hydrodynamics problems (Gardiner & Stone 2005, 2008; Stone et al. 2008).

Work domain: Hydrodynamics/radiative transfer

Working example: `amuse.examples.test_jeans_instability`

Extra requirements/features: MPI parallel

Equivalent codes: Flash, Capreole

Table A.22. Overview of the Core Selection of Codes in AMUSE

Module	Gravitational Dynamics	
Code Name	Application Domain	
ph4	Direct <i>N</i> -body	
SmallN	Direct <i>N</i> -body	
BHTree	Tree-code	
Petar	Hybrid direct-tree <i>N</i> -body	
Mercury	Planetary systems	
SecularMultiple	Hierarchical multiples	
Module		Stellar Evolution
Code Name	Application Domain	
MESA	Heneyy stellar evolution	
SeBa	Parametrized stellar evolution	
Module		Hydrodynamics
Code Name	Application Domain	
Fi	Smoothed-particles Hydrodynamics	
Athena	Grid-based hydrodynamics	
Module		Radiative-transfer Dynamics
Code Name	Application Domain	
Mocassin	Monte Carlo radiative transfer	
SPHray	Monte Carlo radiative transfer	
Module		Chemical Dynamics
Code Name	Application Domain	
Krome	Equilibrium chemistry	
UCLChem	Nonequilibrium chemistry	
Module		Coupling and Initial Conditions
Code Name	Application Domain	
Kepler	Two-body solver	
Kepler_orbiters	<i>N</i> -body planetary solvers	
mmams	Stellar collision routines	
nameclot	Initial conditions generator	
Fastkick	GPU-kick routine for bridge	
FractalCluster	Initial conditions generator	
GalactICS	Initial conditions generator	
Halogen	Initial conditions generator	
hop	Clump finder	

Table A.23. Community Modules in AMUSE, Listed by Domain and Algorithm

GRAVITATIONAL DYNAMICS	
Pure	AarsethZare, Brutus, Hermite, ph4, SmallN.
Direct	Nbody6++, Mercury, HiGPU, Huayno, Rebound,
Approximate	BHTree, Bonsai, Petar.
Dedicated	Kepler, kepler_orbiters, Sakura, TwoBody.
	SecularMultiple, TidyMess.
Limited	Etics.
STELLAR EVOLUTION	
Henrey	EVTwin, MESA.
Parameterized	BSE, SeBa, SSE, CashedSE.
HYDRODYNAMICS	
SPH	Fi, Gadget2
Grid	Athena, Capreole, Flash, MPI-AMRVAC.
Moving mesh	Arepo.
RADIATIVE TRANSFER	
Monte Carlo	Mocassin, SPHray.
Photon transport	SimpleX2.
INTERSTELLAR CHEMISTRY	
Chemistry	Krome. UCLChem.
MISCELLANEOUS	
Analysis	Hop.
Initial conditions	GalactICS, FractalCluster, mameclot.
Potentials	Galactics, HaloGen, Galpy.
Visualization	Vaex, Fresco.
Stellar evolution	Evtwin2Sse.
Disk evolution	Vader.
Planet formation	PACE.
Specialized	ImpactCrater, PPD.
Coupling	Bridge, Venice

Comparable codes: Fi, Gadget2

BHTree (core)

Second-order softened gravitational N -body code with shared time steps and approximate forces. This is a parallel implementation of the original Barnes–Hut tree code (Barnes & Hut [1986](#)).

Work domain: gravitational dynamics, in particular galaxy dynamics
 Working example: `amuse.examples.particles_and_gas_in_cluster`
 Extra requirements/features: MPI parallel, GPU
 Equivalent codes: Bonsai, Octgrav
 Comparable codes: MI6

EVTwin2SSE (core)

Version of EVTwin modified to handle exceptions and hand them to SSE when EVTwin crashes. The handover algorithm performs an rms search on the relative differences in mass, radius, and luminosity between the last known EVTwin state and the entire stellar history of SSE until the first state of a stellar remnant type. A brief description was presented in Section 8.4.3.

Note that although the code supports running several stars simultaneously, the point where EVTwin crashes is strongly dependent on the initial mass of the stars. It is advisable to run the EVTwin2SSE code for one initial mass at a time.

Work domain: Stellar evolution
 Working example: `amuse.examples.hr_diagram_cluster`
 Extra requirements/features: EVTwin, SSE
 Equivalent codes: None
 Comparable codes: None.

Fi (core)

MPI-parallel SPH code (Hernquist & Katz 1989; Pelupessy et al. 2004; Gerritsen & Icke 1997).

Work domain: gravitational dynamics/hydrodynamics
 Working example: `amuse.examples.gas_in_cluster`
 Extra requirements/features: OpenMP parallel
 Equivalent codes: Gadget2
 Comparable codes: Flash, Athena

Flash (core)

Grid-based radiative magnetohydrodynamics solver originally written to simulate nova explosions on white dwarfs (Fryxell et al. 2010). Flash is modular at the compile level, with fundamental structure not dissimilar to AMUSE, using modular codes and providing an environment to combine them.

Work domain: Hydrodynamics, radiative transport
 Working example: None
 Extra requirements/features: OpenMP parallel
 Equivalent codes: Athena
 Comparable codes: Fi, GadGet2

Hermite_GRX (core)

MPI-parallel shared block-time step regularized fourth-order Hermite direct N -body code with post-Newtonian expansions.

Work domain: Collisional relativistic gravitational dynamics, in particular for Galactic nuclei.

Working example: `amuse.examples.montgomery`

Equivalent codes: `hacs64`, `AarsethZare`, `ph4`, `Hermite`

Comparable codes: `Bonsai`, `MI6`, `BHTree`

Huayno (core)

Class containing a large variety of N -body codes based on KDK algorithms using coupled second-order Hamiltonian splitting strategies of tunable order (Pelupessy et al. 2012). Developed for integrating planetary systems using shared or individual time steps.

Work domain: Collisional gravitational dynamics, in particular for planetary systems and star clusters

Working example: `amuse.examples.pythagorean`

Extra requirements/features: MPI parallel, GPU, GRAPE

Equivalent codes: `Hermite`, `Kepler`

Comparable codes: `HiGPUs`, `ph4`, `Hermite`

Kepler (core)

Kepler orbit manipulation functions, imported from Starlab (Portegies Zwart et al. 1999; Hut et al. 2010).

Work domain: Gravitational dynamics/data processing and analysis

Working example: `amuse.examples.kira`

Extra requirements/features: MPI parallel, GPU, GRAPE

Equivalent codes: `Hermite`, `Huayno`, `Kepler`

Comparable codes: `HiGPUs`, `ph4`, `Hermite`

`kepler_orbiters` (core)

Solve Kepler's equations for some number of two-body problems with one shared body, using universal variables (Bate et al. 1971), with a choice between `Sakura` (Gonçalves Ferrari et al. 2014) or `Huayno` (Pelupessy et al. 2012).

Work domain: Gravitational dynamics/data processing and analysis

Working example: `amuse.examples.kepler_orbiters_planets_around_sun`

Extra requirements/features: MPI parallel, GPU, GRAPE

Equivalent codes: `Hermite`, `Kepler`

Comparable codes: `HiGPUs`, `ph4`, `Hermite`

MESA (core)

Heney stellar evolution solver with adaptive mesh refinement (AMR; Paxton et al. 2010, 2011).

Work domain: Stellar evolution

Working example: `amuse.examples.helium_star` see Code example 6.7

Extra requirements/features: OpenMP parallel

Equivalent codes: `EVTwin`

Comparable codes: `SeBa`, `SSE`

MWpotentialBovy2015 (aka galpy) (core)

Static galactic potential model with characteristics similar to the Milky Way (Bovy 2015, 2014). The model includes semianalytic prescriptions for the bulge, disk (Miyamoto & Nagai 1975), and a dark-matter halo (Navarro et al. 1996).

Work domain: Gravitational dynamics, for galactic background potentials

Working example: None

Extra requirements/features: None

Equivalent codes: Galaxia

Comparable codes: GalactICs, FractalCluster, HaloGen

ph4 (core)

MPI-parallel individual block-time step fourth-order Hermite direct N -body code with GPU support.

Work domain: Collisional gravitational dynamics, in particular star clusters

Extra requirements/features: MPI parallel, GPU

Working example: `amuse.examples.evolve_orbit_in_potential`

Equivalent codes: hacs64, phiGRAPE

Comparable codes: Bonsai, MI6

Phantom (core)

Smoothed particles hydrodynamics solver

MPI-parallel SPH code written to perform cosmological structure-formation simulations (Price et al. 2018).

Work domain: Gravitational dynamics/hydrodynamics

Working example: see `Ekster`. Extra requirements/features: MPI parallel

Equivalent codes: Fi, Gadget2.

Comparable codes: Athena, Flash

SeBa (core)

Parameterized stellar evolution code, with heuristic recipes for the evolution of binary stars of arbitrary metallicity (Portegies Zwart & Verbunt 2012). The code includes treatments of wind accretion, mass transfer through Roche-lobe overflow, and the effect of supernova explosions on binary orbital parameters. The code was developed for studying the formation and evolution of high-mass X-ray binaries and compact object mergers (Portegies Zwart & Verbunt 1996; Portegies Zwart & Yungelson 1998), but later expanded to include the prescription of the evolution of white dwarf binaries (Toonen et al. 2012). In a new version, it also provides support for triple evolution (Toonen et al. 2016).

Work domain: Stellar, binary, and triple evolution

Working example: `amuse.examples.evolve_binary`

Extra requirements/features: None

Equivalent codes: BSE

Comparable codes: MESA, EVTwin

SecularMultiple (core)

Secular (orbit-averaged) gravitational dynamics of hierarchical multiple systems composed of nested binary orbits (Hammers & Portegies Zwart 2016); see also Section 4.4.4. Tidal interactions (companion as a single body) and relativistic corrections are incorporated.

Work domain: Gravitational dynamics, in particular planetary systems

Working example: `amuse.examples.hierarchical_quadruple`, and see Section 4.4.4

Extra requirements/features: None

Equivalent codes: Sakura

Comparable codes: Mikkola

SmallN (core)

Shared time step direct N body integrator using a fourth-order time-symmetrized Hermite scheme, and including a modified unperturbed treatment of close approaches. SmallN was first introduced in Starlab (Portegies Zwart et al. 1999; Hut et al. 2010) to handle close encounters and resonant multi-particle interactions.

Work domain: Few-body gravitational dynamics, in particular democratic multiple systems

Working example: `amuse.examples.particles_and_multiples`

Extra requirements/features: `amuse.examples.kira`, `amuse.examples.basic_multiples_si` (see Section 8.7.1)

Equivalent codes: Kepler, Kepler_orbiter, Sakura

Comparable codes: Hermite, HiGPUs, ph4, Hermite

SPHray (core)

Time-dependent Monte Carlo radiative-transfer code (Altay et al. 2011). The code was developed to couple an SPH code with radiative transfer to study the nonequilibrium ionization state (HI, HII, HeI, HeII, HeIII, e) and temperature (internal energy/entropy) of arbitrary density fields (Altay et al. 2008). Table 7.1 presents an overview of the code's application domain.

Work domain: Radiative transfer

Working example: `amuse.examples.iliev_test5`

Extra requirements/features: OpenMP parallel

Equivalent codes: Mocassin, SimpleX

Comparable codes: MPI-AMRVAC, Flash.

Tres (core)

The Triple-star Evolution Code (Toonen et al. [2012](#), [2016](#)). More a coupled AMUSE script than a separate code, combining `SeBa` and `Secular` multiples by including tidal effects, spin evolution, and mass transfer in triples.

Work domain: Triple-star evolution

Extra requirements/features: None

Equivalent codes: None

Comparable codes: None

Envelope selection: These are codes we support, but not explicitly. It is largely up to the user community to help maintain these codes. If you need to use one of these codes, it may be easiest to first find an equivalent alternative from the core selection of codes. If the demand for some of these codes grows, we will consider making them part of the core selection. At this point, few research projects have requested or used these codes, and therefore we cannot spare the effort to maintain them. Our recommendation is to test these codes before usage.

AarsethZare (envelope)

Regularized direct N -body code with a Bulirsch–Stoer (Bulirsch & Stoer [1964](#)) integrator with support for post-Newtonian relativistic corrections to 2.5th order (Aarseth & Zare [1974a](#), [b](#)).

Work domain: Gravitational dynamics, in particular few-body interactions

Working example: None

Extra requirements/features: None

Equivalent codes: `Brutus`, `Hermite_GRX`, `Huayno`

Comparable codes: `MI6`, `SmallN`

Bonsai (envelope)

Second-order softened gravitational N -body code with shared time steps and approximate tree-code forces controlled by an opening angle θ . This code is massively parallel and designed to run concurrently on a large number (tested to up to 18,600) of graphical processing units (Bédorf et al. [2014](#)).

Work domain: Gravitational dynamics, in particular galaxy dynamics

Working example: None

Extra requirements/features: MPI parallel, GPU, parallel-GPU

Equivalent codes from core: `BHTree`, `Octgrav`

Comparable codes: `MI6`

BSE (envelope)

Parameterized stellar evolution code expanded with heuristic recipes for the evolution of binary stars (Hurley et al. [2013a](#)). The code includes treatments for wind accretion, mass transfer through Roche-lobe overflow, and the effect of supernova explosions on binary orbital parameters.

Work domain: Stellar and binary evolution

Working example: None

Extra requirements/features: None

Equivalent codes: SeBa

Comparable codes: MESA, EVTwin

Brutus (envelope)

Direct N -body code using Bulirsch–Stoer integration (Bulirsch & Stoer 1964), with support for arbitrary-precision arithmetic to calculate the orbits of multiple self-gravitating objects (Boekholt & Portegies Zwart 2015). An older version was called `adaptb`.

Work domain: Collisionless gravitational dynamics, in particular few-body systems to high precision and accuracy

Working example: None

Extra requirements/features: MPI parallel, GPU

Equivalent codes: None

Comparable codes: AarsethZare

EVTwin (envelope)

Heney stellar evolution code for all ranges of mass and composition (Eggleton et al. 2011).

Work domain: Stellar and binary evolution

Working example: `amuse.examples.hr_diagram_cluster`

Extra requirements/features: None

Equivalent codes: MESA

Comparable codes: SeBa, SSE, BSE

GadGet2 (envelope)

MPI-parallel SPH code written to perform cosmological structure-formation simulations, but implemented for more general applications (Springel 2000).

Work domain: Gravitational dynamics/hydrodynamics, in particular cosmology

Working example: `amuse.examples.gas_in_cluster`

Extra requirements/features: MPI parallel

Equivalent codes: Fi

Comparable codes: Athena, Flash

Hermite (envelope)

MPI-parallel shared block-time step fourth-order Hermite direct N -body code with GPU support (McMillan 2014).

Work domain: Collisional gravitational dynamics, in particular for star clusters

Working example: `amuse.examples.pythagorean`

Extra requirements/features: MPI parallel, GPU

Equivalent codes: hacs64, phiGRAPE, ph4

Comparable codes: Bonsai, MI6, BHTree

Krome (envelope)

Chemical network for solving the chemodynamical kinetics of a system modeled by coupled ordinary differential equations (Grassi et al. 2014). Incorporated processes include photochemistry, cooling, heating, dust treatment, and reverse kinetics. It was designed to be used for studying primordial chemistry in low-metallicity environments, molecular cloud chemistry, and planetary atmospheres and dust grain populations.

Work domain: Chemical evolution/hydrodynamics

Working example: `amuse.examples.molecular_cloud_chemistry`

Extra requirements/features: None

Equivalent codes: None

Comparable codes: UCLChem

MI6 (envelope)

MPI-parallel block-time step sixth-order Hermite direct N -body code with support for GPU and GRAPE (Iwasawa et al. 2015). It was designed for integrating stellar orbits around a supermassive black hole, and has special treatment for this. It includes post-Newtonian expansions to general relativity up to 3.5th order (gravitational wave kick).

Work domain: Collisional gravitational dynamics, in particular for galactic nuclei

Working example: `amuse.examples.plot_nbody_performance`

Extra requirements/features: MPI parallel, GPU, GRAPE

Equivalent codes: Mikkola, Pikachu (Fujii et al. 2012)

Comparable codes: HiGPUs, ph4, Hermite

Mikkola (envelope)

Direct N -body code for self-gravitating systems, using algorithmic regularization and incorporating post-Newtonian effects (Mikkola 1983; Mikkola & Merritt 2008). The code was designed for integrating stellar orbits around a supermassive black hole. It includes post-Newtonian expansions to general relativity to 2.5th order (gravitational wave radiation).

Work domain: Collisional gravitational dynamics, in particular galactic nuclei

Extra requirements/features: None

Equivalent codes: MI6

Comparable codes: HiGPUs, ph4, Hermite, SmallN

Mercury (envelope)

Symplectic N -body solver dedicated to studying the long-term dynamical evolution of planetary systems (Chambers & Migliorini 1997; Chambers 2012). The equations of motion are solved using a variety of integrators, including mixed-variable symplectic methods (which are in fact quite similar to the Bridge

method discussed in Chapter 9), the Bulirsch–Stoer method, and a hybridization for studying planetary coagulation.

Work domain: Gravitational dynamics, in particular planetary systems

Working example: `amuse.examples.mercury_example`

Extra requirements/features: None

Equivalent codes: *Huayno*, *Sakura*

Comparable codes: *HiGPUs*, *ph4*, *Hermite*, *Kepler*

Mocassin (envelope)

Three-dimensional Monte Carlo photoionization code with radiative transfer through arbitrary geometry and density distributions (Ercolano 2011). The code was originally developed to the model photoionized regions such as HII regions and planetary nebulae (Ritzerveld & Icke 2006). Table 7.1 presents an overview of the code’s application domain.

Work domain: Radiative transfer

Working example: `amuse.examples.h2region`

Extra requirements/features: None

Equivalent codes: *SimpleX2*, *SPHray*

Comparable codes: *MPI-AMRVAC*, *Flash*

Nbody6++ (envelope)

Direct N -body code using a fourth-order Hermite integrator with individual block-time steps and regularized treatment of multiple interactions (Aarseth 2011).

Work domain: Collisional gravitational dynamics, in particular star clusters

Working example: None

Extra requirements/features: MPI parallel, GPU, GRAPE

Equivalent codes: *hacs64*, *MI6*

Comparable codes: *HiGPUs*, *ph4*, *Hermite*

Octgrav (envelope)

MPI-parallel and GPU-enabled tree code up to octupole moments (Gaburov et al. 2010b).

Work domain: Collisionless gravitational dynamics, in particular star clusters

Working example: None

Extra requirements/features: MPI parallel, GPU

Equivalent codes: *BHTree*, *Bonsai*

Comparable codes: *HiGPUs*, *ph4*, *Hermite*

phiGRAPE (envelope)

Block time step fourth-order Hermite direct N -body code with support for GRAPE and/or GPU (Harfst et al. 2007).

Work domain: Collisional gravitational dynamics, in particular star clusters

Extra requirements/features: MPI parallel, GRAPE-4, GRAPE-6, GPU

Equivalent codes: `hacs64`, `ph4`
 Comparable codes: `Bonsai`, `MI6`

PPD (envelope)

The FRIED interpolator object. Performs linear interpolation on the FRIED grid (Haworth et al. [2020](#)).

Work domain: Proto-planetary disks in stellar clusters

Extra requirements/features: None

Equivalent codes: None

Comparable codes: `Mocassin`, `SPHray`

Rebound (envelope)

Symplectic direct N -body integrator designed to study planetary rings, but of more general applicability (Rein & Liu [2012](#); Rein & Tamayo [2018](#); Rein & Liu [2011](#)).

Work domain: Gravitational dynamics, in particular planetary systems

Working example: None

Extra requirements/features: MPI parallel

Equivalent codes: `Huayno`, `Mercury`, `Sakura`

Comparable codes: `ph4`, `Hermite`

Sakura (envelope)

Kepler-based Hamiltonian splitting method for solving the gravitational N -body problem (Gonçalves Ferrari et al. [2014](#)). This is a general N -body code, but due to the imposed data structure it is ideally suited for integrating systems with large binary fractions.

Work domain: Collisional gravitational dynamics, in particular planetary systems and star clusters

Extra requirements/features: MPI parallel

Working example: None

Equivalent codes: `SecularMultiple`, `Huayno`, `Kepler_orbiter`

Comparable codes: `hacs64`, `phiGRAPE`

SimpleX (envelope)

Radiative-transfer solver for an optically thin medium using unstructured Delaunay grid (Paardekooper et al. [2010](#)). Simplex was originally developed for cosmological radiative transfer (Altay et al. [2008](#)). Table [7.1](#) presents an overview of the code's application domain.

Work domain: Radiative transfer

Working example: `amuse.examples.iliev_test5`

Extra requirements/features: OpenMP parallel

Equivalent codes: `Mocassin`, `SPHray`

Comparable codes: `MPI-AMRVAC`, `Flash`

SSE (envelope)

Parameterized stellar evolution code with heuristic recipes for the evolution of single stars with arbitrary metallicity (Hurley et al. [2013b](#)). The code was originally used for studying stellar populations in star clusters (Hurley et al. [2000](#)).

Work domain: single stellar evolution

Working example: None

Extra requirements/features: None

Equivalent codes: BSE, SeBa

Comparable codes: MESA, EVTwin

TPI (envelope)

CUDA code for test-particle integration.

Work domain: Gravitational dynamics, in particular planetesimal or test-particle dynamics

Working example: none. Extra requirements/features: CUDA

Equivalent codes: SmallN, Kepler

Comparable codes: Huayno, Sakura

twobody (envelope)

Python module to solve Kepler's equations for two-body problems using universal variables (Bate et al. [1971](#)).

Work domain: Gravitational dynamics, in particular two-body dynamics

Working example: None

Extra requirements/features: None

Equivalent codes: SmallN, Kepler

Comparable codes: Huayno, Sakura

Vader (envelope)

Viscous disk evolution code (Krumholz & Forbes [2015](#)). We extended the code to include internal photoevaporation with mass-loss profiles. It is built to work with the FRIED grid through the PPD class in AMUSE.

Work domain: Proto-planetary disks in stellar clusters

Extra requirements/features: None

Equivalent codes: None

Comparable codes: Mocassin, SPHray.

Incomplete interface: These codes have been incorporated at some time into AMUSE, but are not longer (or not yet) explicitly supported. We consider them interesting, but they are not often used and we do not have the resources to maintain them. If you plan on using one of these, contact us to see if other users are interested in joining in to improve and complete the interface.

- **Arepo**
 Vorenoi mesh hydrodynamical solver for simulating interstellar gas and solids (Springel [2010a](#)).
 Work domain: Collisional gravitational dynamics with tides
 Working example: None
 Extra requirements/features: MPI parallel
 Equivalent codes: None.
 Comparable codes: Fi, Gadget2, Athena, Flash
- **Capreole**
 MPI-parallel grid-based astrophysical hydrodynamics code (Mellema et al. [1991](#)). Works in one, two, or three spatial dimensions. For the hydrodynamics, it relies on the Roe–Eulderink–Mellema solver, which is an approximate Riemann solver for arbitrary metrics. It can solve different hydrodynamical problems. Capreole has run not only on single processors but also on massively parallel systems (up to 512 processors on BlueGene/L).
 Work domain: Hydrodynamics, in particular cosmology
 Working example: `amuse.examples.linear_wave`
 Extra requirements/features: OpenMP parallel
 Equivalent codes: Athena, Flash,
 Comparable codes: Fi, GadGet2
- **Etics**
 Gravity solver for collisionless stellar dynamical simulations using two fast force calculation methods based on series expansions of the Poisson equation (Meiron et al. [2014](#)). One method is the self-consistent field model, a Fourier-like expansion of the density field in some basis set. The other method is a multipole expansion method, a Taylor-like expansion of the Green’s function.
 Work domain: Collisionless gravitational dynamics, in particular elliptical galaxies
 Working example: None
 Extra requirements/features: MPI parallel, GPU
 Equivalent codes: None
 Comparable codes: Bonsai, Octgrav
- **HiGPUs**
 Block-time step sixth-order Hermite direct gravitational N -body code (Spera et al. [2012](#)).
 Work domain: Collisional gravitational dynamics, in particular star clusters or planetary systems
 Working example: None
 Extra requirements/features: MPI parallel, GPU
 Equivalent codes: MI6
 Comparable codes: ph4, Hermite
- **hacs64**
 Block time step Hermite direct N -body code with Ahmad–Cohen neighbor scheme and GPU support. The code uses a sixth-order Hermite integrator for the irregular step and a fourth-order Hermite for the regular step.

Work domain: Collisional gravitational dynamics

Working example: None

Extra requirements/features: GPU

Equivalent codes: MI6

Comparable codes: HiGPUs, ph4, Hermite

- **ImpactCrater**

Impact crater size calculator. Simulates the effect of an impact of a minor body on a rocky major body.

Work domain: Collisional dynamics

Working example: None

Extra requirements/features: None

Equivalent codes: none

Comparable codes: Fi, Gadget2, Phantom, Arepo

- **MPI-AMRVAC**

Three-dimensional magnetohydrodynamics AMR code with radiative transfer (van der Holst et al. 2012; Xia et al. 2018),⁴ using the versatile advection method (Tóth & Keppens 2012). The code was developed to simulate shock-dominated problems (Meliani et al. 2007; Keppens & Meliani 2012), but handles smooth flows as well (Porth et al. 2014).

Work domain: Hydrodynamics

Working example: `amuse.examples.linear_wave_boundaries`

Extra requirements/features: MPI parallel

Equivalent codes: Flash

Comparable codes: Athena

- **PACE**

Planet population synthesis code for simulating the formation of planetary systems from photoevaporated (PPD) viscous (vader) disks (Huang et al. 2024).

Work domain: Planet formation

Working example: None

Extra requirements/features: Venice, Vader, PPD

Equivalent codes: None

Comparable codes: None

- **Ramses (deprecated)**

AMR hydrodynamics solver with a tree-based data structure allowing recursive grid refinements on a cell-by-cell basis (Teyssier 2010). Based on the method of Kravtsov et al. (1997) and designed to study structure formation in the Universe (Teyssier 2002).

Work domain: Hydrodynamics/radiative transfer, in particular cosmology

Working example: `amuse.examples.linear_wave_boundaries`

Extra requirements/features: None

⁴ See also <http://amrvac.org/> or the github version at <https://github.com/amrvac/amrvac>.

Equivalent codes: Flash, Athena, Capreole, MPI-AMRVAC

Comparable codes: Fi, Gadget2

- Tupan

Direct N -body code for collisional stellar dynamics. Like Huayno, it contains a wide range of different integrators. The code was originally written by Guilherme Goncalves, but regrettably never published.

Work domain: Collisional gravitational dynamics

Working example: None

Extra requirements/features: MPI parallel, GPU

Equivalent codes: Huayno, Sakura, Sei

Comparable codes: Kepler

- TidyMess

Direct N -body code with tidal evolution solver. Excellently suited for chaotic N -body problems for which tides are relevant (Boekholt & Correia 2023).

Work domain: Collisional gravitational dynamics with tides

Working example: None

Extra requirements/features: MPI parallel

Equivalent codes: Huayno, Sakura, Sei

Comparable codes: Kepler

- Venice

General solver for matrices of differential equations, primarily used for bridging multiple codes (Wilhelm & Portegies Zwart 2024)

Work domain: General multicode bridging strategy

Working example: see Section 10.1.3

Extra requirements/features: None

Equivalent codes: None

Comparable codes: PACE

Support codes: These are codes for

- FractalCluster

Generator for producing fractal stellar distributions by defining a cube of side 2 which is subsequently divided into subcubes, on which the same star addition algorithm is used. Eventually, this procedure leads to a three-dimensional, scale-free distribution of equal-mass particles (Goodwin & Whitworth 2004).

Work domain: Initial conditions

Working example: `amuse.examples.plot_fractal_clumpy_cluster`

Extra requirements/features: None

Equivalent codes: None

Comparable codes: GalactICsHaloGen

- **Fresco**
 Visualization package specific for making “observational” images of simulations, focused on simulations in which stellar evolution, stellar dynamics, and possibly gas dynamics are incorporated. *Fresco* is not on the *amuse* github page, but can be downloaded from github at <https://github.com/rieder/Fresco>.
 Work domain: Visualization
 Working example: None
 Extra requirements/features: None
 Equivalent codes: None
 Comparable codes: *vaex*
- **GalactICs**
 Generator for an N -body particle realization of an axisymmetric galaxy that consists of a disk, bulge, and halo (Kuijken & Dubinski 2011), based on the original work by Kuijken & Dubinski (1995).
 Work domain: Initial conditions
 Working example: None
 Extra requirements/features: None
 Equivalent codes: *FractalCluster*, *HaloGen*
 Comparable codes: *Galaxia*
- **Galaxia**
 Dynamic potential of a galactic model with characteristics similar to the Milky Way (Sharma et al. 2011). The model includes semianalytic prescriptions for the axisymmetric parts (bulge, disk, halo; Allen & Santillan 1991), a parameterization for the central (Ferrers) bar (Romero-Gómez et al. 2007, 2011) and transient two-dimensional and three-dimensional spiral arms (Cox & Gómez 2002).
 Work domain: Gravitational dynamics, in particular for background potentials
 Working example: None
 Extra requirements/features: None
 Equivalent codes: *MWpotentialBovy2015*
 Comparable codes: *GalactICs*, *FractalCluster*, *HaloGen*
- **HaloGen**
 Generator for an N -body particle realization of the Galactic halo (Zemp et al. 2008; Zemp 2014; Avila Perez & Murray 2015) by generating cosmological tracer distributions (halos) for a range of different cosmologies.
 Work domain: Initial conditions, in particular for galaxies
 Working example: `amuse.examples.solar_cluster_in_live_galaxy`
 Extra requirements/features: Parallel
 Equivalent codes: *GalactICs*, *FractalCluster*.
 Comparable codes: *Galaxia*

- **hop**
 Coordinate-free group-finding algorithm for particle distributions in three dimensions (Eisenstein & Hut 2011). Original use was finding groups of particles in N -body realizations in Cartesian coordinates.
 Work domain: Data processing and analysis
 Working example: `amuse.examples.clump_finding`
 Extra requirements/features: None
 Equivalent codes: None
 Comparable codes: None
- **mameclot**
 Generator for an N -body particle realization of one or two star clusters in orbit around each other (see <https://github.com/mgieles/mameclot>). The velocities can be isotropic or radially anisotropic (Osipkov 1979; Merritt 1985).
 Work domain: Initial conditions
 Working example: None
 Extra requirements/features: None
 Equivalent codes: None
 Comparable codes: `FractalCluster`, `GalactICs`
- **mmams**
 Fast stellar collision method using Archimedes’s principle to merge stars, including rudimentary treatments for shock heating, hydrodynamic mixing, mass ejection, and angular momentum transfer (Gaburov et al. 2008; Lombardi & Sawyer Warren 2014).
 Work domain: Resolving stellar collisions
 Working example: See Code example 8.10
 Extra requirements/features: None
 Equivalent codes: None
 Comparable codes: `Gadget2`, `Fi`
- **MMC**
 Monte Carlo star cluster dynamics simulation code (Askar et al. 2017). The code was developed to compare simulation results with observed globular clusters (Giersz 1998, 2001).
 Work domain: Gravitational dynamics
 Working example: None
 Extra requirements/features: None
 Equivalent codes: None
 Comparable codes: None
- **UCLChem**
 Chemical network for solving the chemodynamical kinetics of a system modeled by coupled ordinary differential equations (Holdship et al. 2017; Keil et al. 2022). Incorporated processes include photochemistry, cooling, heating, dust treatment, and reverse kinetics. It was designed to be used for studying primordial chemistry in low-metallicity environments, molecular cloud chemistry, and planetary atmospheres and dust grain populations.

Work domain: Chemical evolution/hydrodynamics

Working example: None

Extra requirements/features: None

Equivalent codes: None

Comparable codes: Krome

- **Vaex**

Visualization package written in Python for analyzing large data volumes. It was written by Maarten Breddels as a project at Groningen University (Breddels 2017), see also <https://vaex.io>.

Work domain: Visualization and analysis

Working example: None

Extra requirements/features: Does not require but can use the GPU

Equivalent codes: None

Comparable codes: `pyplot`, `gnuplot`

We are sorry if your code is not among the listed modules, and we will gladly work with you to include it. Writing a custom interface to a new code is relatively easy, but, regrettably, this is not the right place to explain how to develop a new AMUSE interface. However, we can teach you how to do this, or you may want to visit the amusecode.org website for a brief tutorial.

A.8.3 Packages Removed Since the 2018 Version of the Book

Regrettably, we cannot muster the support we would love to provide for all packages to which AMUSE has an interface. Some of these packages have similar functionality as others, and some are simply not used a lot. The result is that bit-rot sets in, compilers change, and it turns out that modules are not always written in a sustainable way. Most FORTRAN code remains compilable, but for many C and C++ packages this is not the case. In addition, some of these codes have become obsolete, as new versions or better implementations of the same processes have become available. So long as no new bugs are found, or if the original authors remain accessible, this does not necessarily pose a problem. However, not using a code for a while does not stop compilers and computers from changing, and therefore the codes may stop working eventually.

An example is the code `Adaptb`, which basically is an early version of `Brutus` (Boekholt & Portegies Zwart 2015). The code `mmc` (Leveque et al. 2022) was never properly incorporated into the AMUSE framework. We will miss the GPU-supported `treecode` in `Octgrav` (Gaburov et al. 2010b, a), but this was simply replaced by `Bonsai` (Bédorf et al. 2014). The `pikachu` code (Iwasawa et al. 2015) was replaced by `PeTar` (Wang et al. 2015). Although `hacs64` has a sixth-order predictor-corrector Hermite integrator for direct N -body with an Ahmad–Cohen scheme (Ahmad & Cohen 1973) running on GPU, a combination of methods nowhere else available in AMUSE, there are high-order integrators in `Symple` and we can relatively easily expand `ph4` to include higher orders. Eventually, `sei` (Rein

& Tremaine 2011) also had to go, even though it is a nice and effective 183-line direct N -body code; it was replaced with Rebound (Rein & Spiegel 2015), which utilizes the faster IAS15 integrator.

References

- Aarseth, S. J., & Zare, K. 1974a, *CeMec*, 10, 185
- Aarseth, S. J., & Zare, K. 1974b, *CeMec*, 10, 516
- Aarseth, S. J. 2011, NBODY Codes: Numerical Simulations of Many-body (N-body) Gravitational Interactions, Astrophysics Source Code Library, ascl:1102.006
- Ahmad, A., & Cohen, L. 1973, *ApJ*, 179, 885
- Allen, C., & Santillan, A. 1991, *RMxAA*, 22, 255
- Altay, G., Croft, R. A. C., & Pelupessy, I. 2008, *MNRAS*, 386, 1931
- Altay, G., Croft, R. A. C., & Pelupessy, I. 2011, SPHRAY: A Smoothed Particle Hydrodynamics Ray Tracer for Radiative Transfer, Astrophysics Source Code Library, ascl:1103.009
- Askar, A., Giersz, M., Pych, W., & Dalessandro, E. 2017, COCOA: Simulating Observations of Star Cluster Simulations, Astrophysics Source Code Library, ascl:1703.002
- Avila Perez, S., & Murray, S. 2015, HALOGEN: Approximate synthetic halo catalog generator, Astrophysics Source Code Library, ascl:1505.017
- Barnes, J., & Hut, P. 1986, *Nature*, 324, 446
- Bate, R. R., Mueller, D. D., & White, J. E. 1971, Fundamentals of Astrodynamics (New York: Dover)
- Bédorf, J., Gaburov, E., Fujii, M. S., et al. 2014, Proc. of the International Conference for High Performance Computing, Networking, Storage and Analysis, SC '14 (Piscataway, NJ: IEEE) 54
- Boekholt, T., & Portegies Zwart, S. 2015, *ComAC*, 2, 2
- Boekholt, T. C. N., & Correia, A. C. M. 2023, *MNRAS*, 522, 2885
- Bovy, J. 2014, galpy: Galactic Dynamics Package, Astrophysics Source Code Library, ascl:1411.008
- Bovy, J. 2015, *ApJS*, 216, 29
- Breddels, M. A. 2017, IAU Symp 325, Astrodynamics ed. M. Brescia, S. G. Djorgovski, E. D. Feigelson, G. Longo, & S. Cavuoti (Cambridge: Cambridge Univ. Press) 299
- Bulirsch, R., & Stoer, J. 1964, *NuMat*, 6, 413
- Chambers, J. E., & Migliorini, F. 1997, AAS/DPS Meeting, 29, 27.06
- Chambers, J. E. 2012, Mercury: A Software Package for Orbital Dynamics, Astrophysics Source Code Library, ascl:1201.008
- Cox, D. P., & Gómez, G. C. 2002, *ApJS*, 142, 261
- Eggleton, P. P., Tout, C., Pols, O., et al. 2011, STARS: A Stellar Evolution Code, Astrophysics Source Code Library, ascl:1107.008
- Einstein, A. 1915a, SPAW, 778,
- Einstein, A. 1915b, SPAW 799,
- Einstein, A., & Grossmann, M. 1914, *ZMP*, 63, 215
- Eisenstein, D., & Hut, P. 2011, HOP: A Group-finding Algorithm for N-body Simulations, Astrophysics Source Code Library, ascl:1102.019
- Emden, R. 1907, Gaskugeln: Anwendungen der mechanischen Wärmetheorie auf kosmologische und meteorologische Probleme (Leipzig: B. G. Teubner)

- Ercolano, B. 2011, MOCASSIN: MONte CARlo SimulationS of Ionized Nebulae, Astrophysics Source Code Library, ascl:[1110.010](#)
- Euler, L. 1768, *Institutionum Calculi Integralis* (1-3,; St. Petersburg: Academy of Sciences)
- Fryxell, B., Olson, K., Ricker, P., et al. 2010, FLASH: Adaptive Mesh Hydrodynamics Code for Modeling Astrophysical Thermonuclear Flashes, Astrophysics Source Code Library, ascl:[1010.082](#)
- Fujii, M. S., Iwasawa, M., Saitoh, T. R., & Portegies Zwart, S. 2012, ASP Conf. Ser. 453, *Advances in Computational Astrophysics: Methods, Tools, and Outcome* ed. R. Capuzzo-Dolcetta, M. Limongi, & A. Tornambè (San Francisco, CA: ASP)
- Fujii, M., Iwasawa, M., Funato, Y., & Makino, J. 2007, [PASJ](#), **59**, 1095
- Gaburov, E., Lombardi, J. C., & Portegies Zwart, S. 2008, [MNRAS](#), **383**, L5
- Gaburov, E., Bédorf, J., & Portegies Zwart, S. 2010a, [Procedia Computer Science](#), **1**, 1119
- Gaburov, E., Bédorf, J., & Portegies Zwart, S. 2010b, OCTGRAV: Sparse Octree Gravitational N-body Code on Graphics Processing Units, Astrophysics Source Code Library, ascl:[1010.048](#)
- Gaburov, E., Lombardi, J. C., Portegies Zwart, S., & Rasio, F. A. 2018, StarSmasher: Smoothed Particle Hydrodynamics Code for Smashing Stars and Planets, Astrophysics Source Code Library, ascl:[1805.010](#)
- Gardiner, T. A., & Stone, J. M. 2005, [JCoPh](#), **205**, 509
- Gardiner, T. A., & Stone, J. M. 2008, [JCoPh](#), **227**, 4123
- Gear, C. W. 1967, *MaCom*, **21**, 146
- Gerritsen, J. P. E., & Icke, V. 1997, *A&A*, **325**, 972
- Giersz, M. 1998, [MNRAS](#), **298**, 1239
- Giersz, M. 2001, [MNRAS](#), **324**, 218
- Gonçalves Ferrari, G., Boekholt, T., & Portegies Zwart, S. F. 2014, [MNRAS](#), **440**, 719
- Goodwin, S. P., & Whitworth, A. P. 2004, [A&A](#), **413**, 929
- Grassi, T., Bovino, S., Prieto, J., et al. 2014, KROME: Chemistry package for astrophysical simulations, Astrophysics Source Code Library, ascl:[1402.011](#)
- Hairer, E., Lubich, C., & Wanner, G. 2006, *Geometric Numerical Integration: Structure-Preserving Algorithms for Ordinary Differential Equations* (2nd ed.; Dordrecht: Springer)
- Hamers, A. S., & Portegies Zwart, S. F. 2016, [MNRAS](#), **459**, 2827
- Harfst, S., Gualandris, A., Merritt, D., et al. 2007, [NewA](#), **12**, 357
- Haworth, T. J., Cadman, J., Meru, F., et al. 2020, [MNRAS](#), **494**, 4130
- Hernquist, L., & Katz, N. 1989, [ApJS](#), **70**, 419
- Hilbert, D. 1915, *Gott. Nachr.*, **27**, 395
- Holdship, J., Viti, S., Jiménez-Serra, I., Makrymallis, A., & Priestley, F. 2017, [AJ](#), **154**, 38
- Huang, S., Portegies Zwart, S., & Wilhelm, M. J. C. 2024, [A&A](#), **689**, A338
- Hunter, J. D. 2007, [CSE](#), **9**, 90
- Hurley, J. R., Pols, O. R., & Tout, C. A. 2000, [MNRAS](#), **315**, 543
- Hurley, J. R., Tout, C. A., & Pols, O. R. 2013a, BSE: Binary Star Evolution, Astrophysics Source Code Library, ascl:[1303.014](#)
- Hurley, J. R., Pols, O. R., & Tout, C. A. 2013b, SSE: Single Star Evolution, Astrophysics Source Code Library, ascl:[1303.015](#)
- Hut, P., McMillan, S., Makino, J., & Portegies Zwart, S. 2010, Starlab: A Software Environment for Collisional Stellar Dynamics, Astrophysics Source Code Library, ascl:[1010.076](#)
- Iwasawa, M., Portegies Zwart, S., & Makino, J. 2015, [ComAC](#), **2**, 6

- Jiao, Y., Dai, G., Feng, Q., & Tang, Y. 2007, JCM, [25](#), 690
- Keil, M., Viti, S., & Holdship, J. 2022, [ApJ](#), [927](#), 203
- Keppens, R., Meliani, Z., van Marle, A. J., et al. 2012, [JCoPh](#), [231](#), 718
- Kerr, R. P. 1963, [PhRvL](#), [11](#), 237
- Kluyver, T., Ragan-Kelley, B., Perez, F., et al. 2016, [Positioning and Power in Academic Publishing: Players, Agents and Agendas](#), ed. F. Loizides, & B. Schmidt (Amsterdam: IOS Press) 87
- Kozai, Y. 1962, [AJ](#), [67](#), 591
- Kravtsov, A. V., Klypin, A. A., & Khokhlov, A. M. 1997, [ApJS](#), [111](#), 73
- Krumholz, M. R., & Forbes, J. C. 2015, [A&C](#), [11](#), 1
- Kuijken, K., & Dubinski, J. 1995, [MNRAS](#), [277](#), 1341
- Kuijken, K., & Dubinski, J. 2011, GalactICS: Galaxy Model Building Package, Astrophysics Source Code Library, ascl:1109.011
- Lane, J. H. 1870, AmJSA, [50](#), 57
- Leveque, A., Giersz, M., Banerjee, S., et al. 2022, [MNRAS](#), [514](#), 5739
- Lidov, M. L. 1962, [Planet. Space Sci.](#), [9](#), 719
- Lombardi, J., & Sawyer Warren, J. 2014, MMAS: Make Me A Star, Astrophysics Source Code Library, ascl:1412.010
- McMillan, S. L. W. 2014, AAS/DDA Meeting, [45](#), 303.01
- Meiron, Y., Li, B., Holley-Bockelmann, K., & Spurzem, R. 2014, [ApJ](#), [792](#), 98
- Meliani, Z., Keppens, R., Casse, F., & Giannios, D. 2007, [MNRAS](#), [376](#), 1189
- Mellema, G., Eulderink, F., & Icke, V. 1991, A&A, [252](#), 718
- Merritt, D. 1985, [AJ](#), [90](#), 1027
- Mewes, V., Zlochower, Y., Campanelli, M., et al. 2018, PhRvD, [97](#), 084059
- Mewes, V., Zlochower, Y., Campanelli, M., et al. 2020, PhRvD, [101](#), 104007
- Mikkola, S. 1983, [MNRAS](#), [203](#), 1107
- Mikkola, S., & Merritt, D. 2008, [AJ](#), [135](#), 2398
- Miyamoto, M., & Nagai, R. 1975, [PASJ](#), [27](#), 533
- Navarro, J. F., Frenk, C. S., & White, S. D. M. 1996, [ApJ](#), [462](#), 563
- Osipkov, L. P. 1979, PAZh, [5](#), 77
- Paardekooper, J. P., Kruip, C. J. H., & Icke, V. 2010, [A&A](#), [515](#), A79
- Paxton, B., Bildsten, L., Dotter, A., et al. 2010, MESA: Modules for Experiments in Stellar Astrophysics, Astrophysics Source Code Library, ascl:1010.083
- Paxton, B., Bildsten, L., Dotter, A., et al. 2011, [ApJS](#), [192](#), 3
- Paxton, B., Cantiello, M., Arras, P., et al. 2013, [ApJS](#), [208](#), 4
- Paxton, B., Marchant, P., Schwab, J., et al. 2015, [ApJS](#), [220](#), 15
- Paxton, B., Schwab, J., Bauer, E. B., et al. 2018, [ApJS](#), [234](#), 34
- Pelupessy, F. I., van der Werf, P. P., & Icke, V. 2004, [A&A](#), [422](#), 55
- Pelupessy, F. I., Jänes, J., & Portegies Zwart, S. 2012, [NewA](#), [17](#), 711
- Plummer, H. C. 1911, [MNRAS](#), [71](#), 460
- Poisson, E., & Will, C. M. 2014, Gravity (Cambridge: Cambridge Univ. Press)
- Portegies Zwart, S. F., & Verbunt, F. 1996, A&A, [309](#), 179
- Portegies Zwart, S. F., & Verbunt, F. 2012, SeBa: Stellar and Binary Evolution, Astrophysics Source Code Library, ascl:1201.003
- Portegies Zwart, S. F., & Yungelson, L. R. 1998, [A&A](#), [332](#), 173
- Portegies Zwart, S. F., Makino, J., McMillan, S. L. W., & Hut, P. 1999, [A&A](#), [348](#), 117

- Portegies Zwart, S. F., Boekholt, T. C. N., Por, E. H., Hamers, A. S., & McMillan, S. L. W. 2022, [A&A](#), **659**, [A86](#)
- Portegies Zwart, S. 2018, [Sci.](#), **361**, [979](#)
- Portegies Zwart, S. F., & Boekholt, T. C. N. 2018, [CNSNS](#), **61**, [160](#)
- Porth, O., Xia, C., Hendrix, T., Moschou, S. P., & Keppens, R. 2014, [ApJS](#), **214**, [4](#)
- Price, D. J., Wurster, J., Tricco, T. S., et al. 2018, [PASA](#), **35**, [e031](#)
- Rein, H., & Liu, S. F. 2012, [A&A](#), **537**, [A128](#)
- Rein, H., & Liu, S.-F. 2011, REBOUND: Multi-purpose N-body Code for Collisional Dynamics, Astrophysics Source Code Library, [ascl:1110.016](#)
- Rein, H., & Spiegel, D. S. 2015, [MNRAS](#), **446**, [1424](#)
- Rein, H., & Tamayo, D. 2018, [MNRAS](#), **473**, [3351](#)
- Rein, H., & Tremaine, S. 2011, [MNRAS](#), **415**, [3168](#)
- Ritzerveld, J., & Icke, V. 2006, [PhRvE](#), **74**, [026704](#)
- Romero-Gómez, M., Athanassoula, E., Masdemont, J. J., & García-Gómez, C. 2007, [A&A](#), **472**, [63](#)
- Romero-Gómez, M., Athanassoula, E., Antoja, T., & Figueras, F. 2011, [MNRAS](#), **418**, [1176](#)
- Schwarzschild, K. 1916, Sitzungsber. Königl. Preuss. Akad. Wissenschaften, **7**, 189
- Sharma, S., Bland-Hawthorn, J., Johnston, K. V., & Binney, J. 2011, Galaxia: A Code to Generate a Synthetic Survey of the Milky Way, Astrophysics Source Code Library, [ascl:1101.007](#)
- Sobol, I. M. 1967, ZVMMF, **7**, [784](#)
- Sofroniou, M., & Spaletta, G. 2005, [Optimization Methods and Software](#), **20**, [597](#)
- Spera, M., Capuzzo Dolcetta, R., & Punzo, D. 2012, HiGPUs: Hermite's N-body Integrator Running on Graphic Processing Units, Astrophysics Source Code Library, [ascl:1207.002](#)
- Springel, V. 2000, GADGET-2: A Code for Cosmological Simulations of Structure Formation, Astrophysics Source Code Library, [ascl:0003.001](#)
- Springel, V. 2010a, [MNRAS](#), **401**, [791](#)
- Springel, V. 2010b, [ARA&A](#), **48**, [391](#)
- Stone, J. M., Gardiner, T. A., Teuben, P., Hawley, J. F., & Simon, J. B. 2008, [ApJS](#), **178**, [137](#)
- Stone, J. M., Gardiner, T. A., Teuben, P., Hawley, J. F., & Simon, J. B. 2010, Athena: Grid-based Code for Astrophysical Magnetohydrodynamics (MHD), Astrophysics Source Code Library, [ascl:1010.014](#)
- Teyssier, R. 2002, [A&A](#), **385**, [337](#)
- Teyssier, R. 2010, RAMSES: A New N-body and Hydrodynamical Code, Astrophysics Source Code Library, [ascl:1011.007](#)
- Toonen, S., Nelemans, G., & Portegies Zwart, S. 2012, [A&A](#), **546**, [A70](#)
- Toonen, S., Hamers, A., & Portegies Zwart, S. 2016, [ComAC](#), **3**, [6](#)
- Tóth, G., & Keppens, R. 2012, VAC: Versatile Advection Code, Astrophysics Source Code Library, [ascl:1207.003](#)
- van der Holst, B., Keppens, R., Meliani, Z., et al. 2012, MPI-AMRVAC: MPI-Adaptive Mesh Refinement-Versatile Advection Code, Astrophysics Source Code Library, [record ascl:1208.014](#)
- Vasiliev, E. 2019, [MNRAS](#), **482**, [1525](#)
- Vaught, A. 1996, Linux J., 1996, 7–es
- Wang, L., Spurzem, R., Aarseth, S., et al. 2015, [MNRAS](#), **450**, [4070](#)
- Weinstein, G. 2012, arXiv:[1202.2791](#)

- Wilhelm, M. J. C., & Portegies Zwart, S. 2024, *A&A*, 691, A71
- Williams, T., Kelley, C., et al. 2013, Gnuplot 4.6: An Interactive Plotting Program, <http://gnuplot.sourceforge.net/>.
- Xia, C., Teunissen, J., El Mellah, I., Chané, E., & Keppens, R. 2018, *ApJS*, 234, 30
- Yoshida, H. 1990, *PhLA*, 150, 262
- Zemp, M. 2014, Halogen: Multimass Spherical Structure Models for N-body Simulations, Astrophysics Source Code Library, ascl:1407.020
- Zemp, M., Moore, B., Stadel, J., Carollo, C. M., & Madau, P. 2008, *MNRAS*, 386, 1543
- Zhao, Y., Zhang, Y., & Wang, J. 2021, International Journal of Advanced Network, Monitoring and Controls, 6, 20

QUERY FORM

BOOK TITLE: Astrophysical Recipes (Second Edition)

AUTHOR: Simon Portegies Zwart, Steve McMillan and Steven Rieder

CHAPTER TITLE: Back matter

Page 38

Q1

Please provide citation for Table A.22.

add reference to first paragraph of section A.8